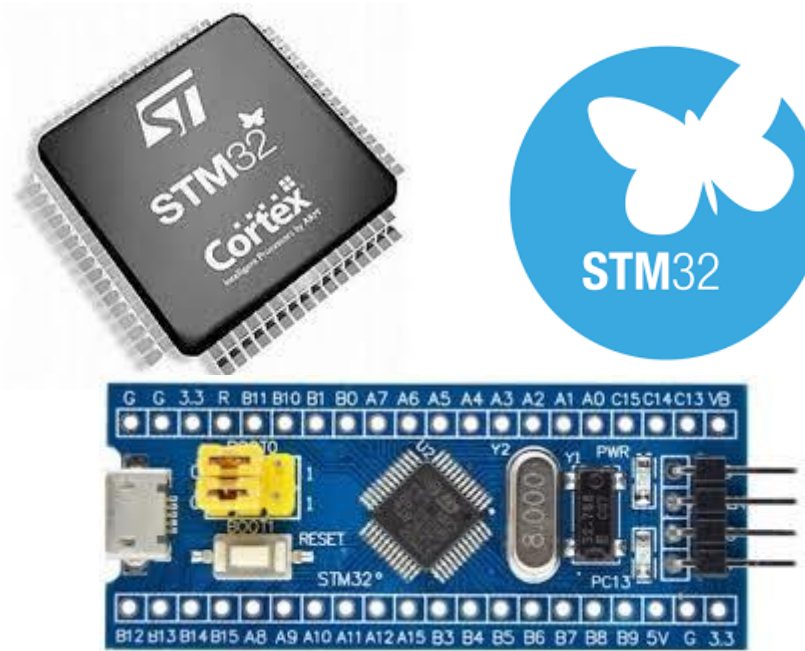


Cours Architecture des systèmes électroniques embarqués

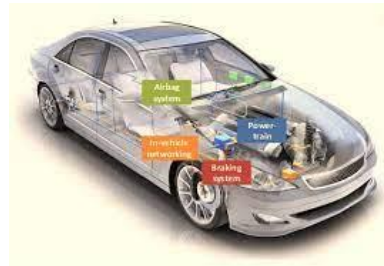


STM32

Introduction

Chaque jour nous utilisons des produit qui contient des puces d'ordinateur qui sont constitués d'une UC (CPU),des mémoires et des programmes.

Real Life examples of Embedded systems

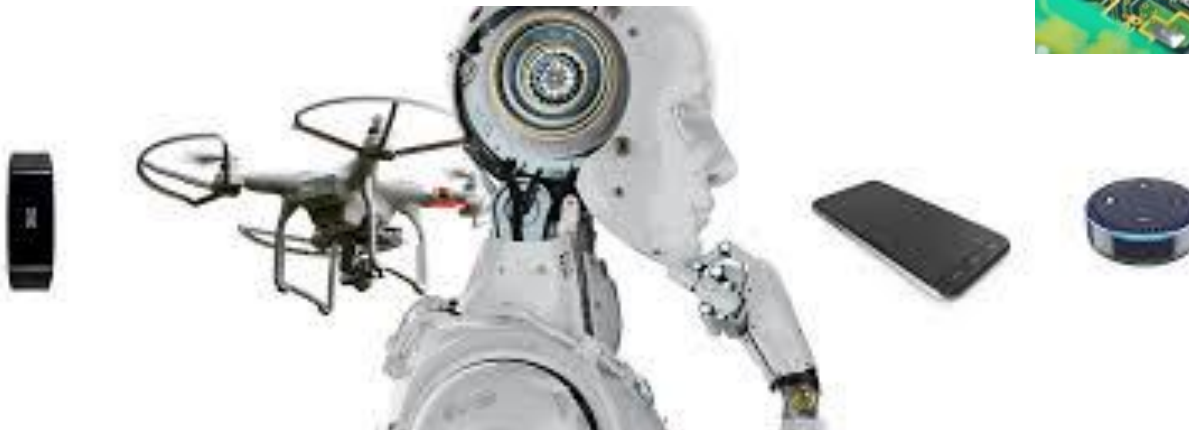


Les Systèmes Embarqués

Embedded systems

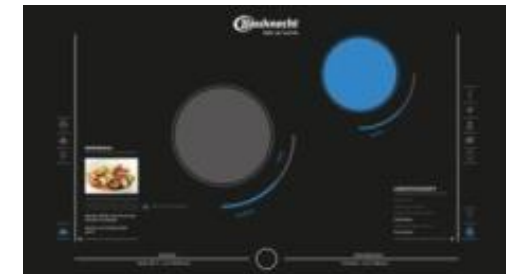
«Un système embarqué est un système électronique et informatique autonome, souvent temps réel, spécialisé dans une tâche précise. Le terme désigne aussi bien le matériel informatique que le logiciel utilisé. »

(Source : Wikipédia)



Les Systèmes Embarqués

- Un système embarqué n'est **pas** forcément:
 - mobile,
 - petit,
 - caché,
 - puissant (ou non puissant),
 - complexe,
 - alimenté par batterie,
 - économe en énergie,
 - cher (ou bon marché),
 - spécifique,
 - industriel,
 - ...



Principales caractéristiques des SE

- Encombrement mémoire
- Consommation d'énergie
- Poids et volume
- Criticité / Fiabilité / Tolérance aux fautes
- Fonctionnel. Le système doit toujours fonctionner correctement.
- Sûr, surtout si la sécurité des personnes est en jeu.
- Interfaçage avec monde physique
- Contraintes environnementales
- Contraintes temps réel
- Coûts
- Système principalement numérique
- Fiables : destinés à un fonctionnement **autonomes** qui ne tolère pas l'erreur.
- ...

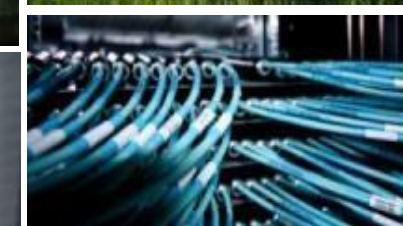


Source : NVidia

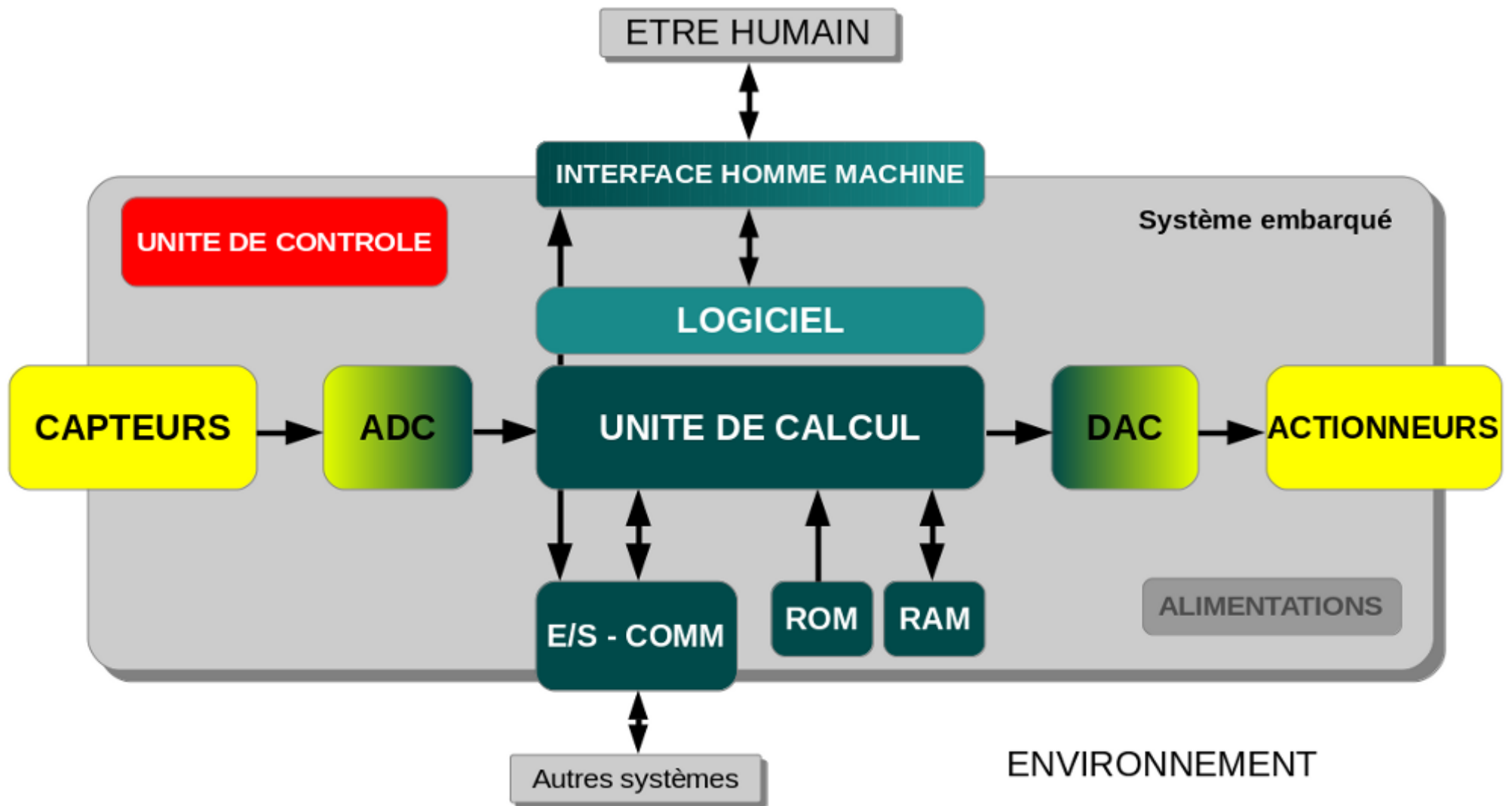


Principaux domaines d'application des SE

- **Transports**
 - Aéronautique
 - Automobile / Camions / Machinisme agricole
 - Ferroviaire
 - ...
- Spatial
- Militaire / Défense
- Réseaux / Télécommunications
- Domotique / Bâtiments
- Distribution et gestion de l'énergie
- Médical
- Terminaux de paiement
- Consumer electronics
- Robotique
- ...



Architecture d'un Système Embarqué

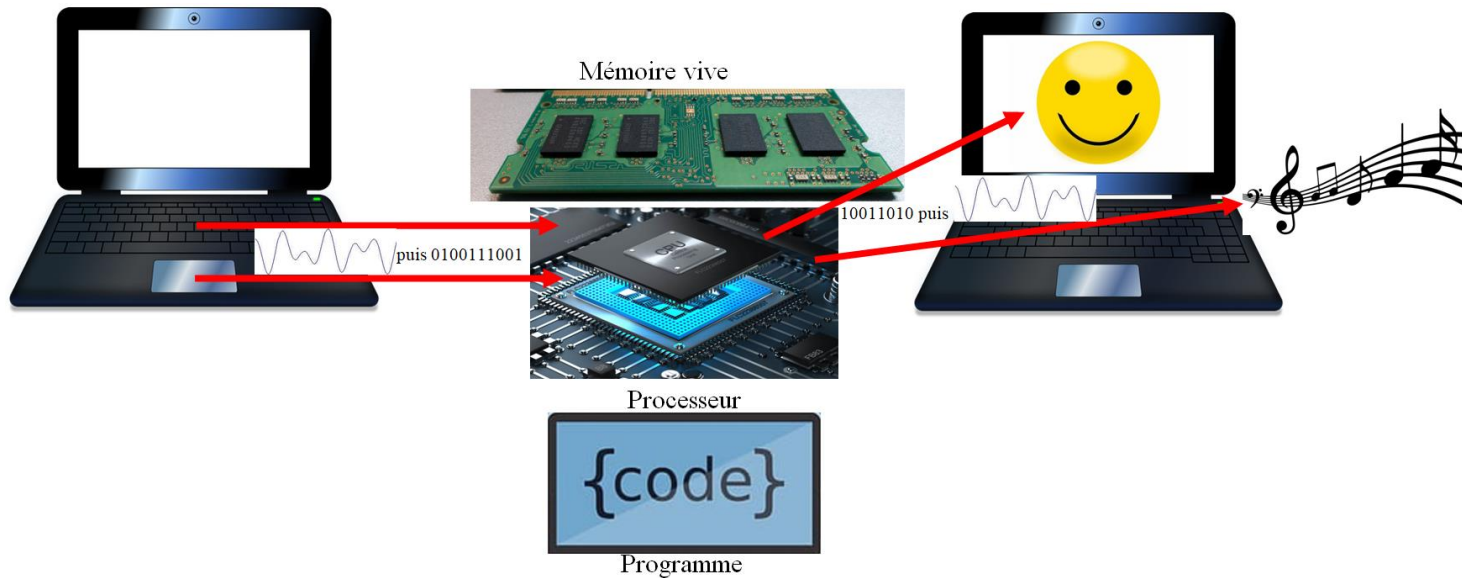


Architecture d'un Système Embarqué

- 1. Unité de calcul (CPU)** est le cerveau du système embarqué, il exécute toutes les opérations du système
- 2. Mémoire** : La mémoire est utilisée pour stocker les données, les programmes, les paramètres et d'autres informations essentielles pour le fonctionnement du système. Elle se divise souvent en deux types :
 - Mémoire vive (RAM)** : Utilisée pour le stockage temporaire des données en cours d'utilisation.
 - Mémoire morte (ROM ou Flash)** : Stocke les programmes et les données permanentes.
- 3. Capteur** : Il mesure la quantité physique et la convertit en un signal électrique qui peut être lu par un observateur ou par n'importe quel instrument électronique comme un convertisseur A/N. Le capteur stocke la quantité mesurée dans la mémoire.
- 4. Actionneur** : Un actionneur compare la sortie fournie par le convertisseur (N/A) à la sortie réelle (attendue) qui y est stockée et stocke la sortie approuvée.
- 5. Convertisseur Analogique/Numérique (ADC)** : Un convertisseur (A/N) convertit le signal analogique envoyé par le capteur en un signal numérique.
- 6. Convertisseur Numérique/Analogique (N/A)** : Un convertisseur numérique-analogique convertit les données numériques fournies par le processeur en données analogiques.

Architecture d'un Système Embarqué

Exemple



Lorsque vous utilisez votre ordinateur portable, vous saisissez des informations sur le clavier ou déplacez votre doigt sur le pavé tactile.

- Chaque appui est capté puis traduit en signaux électriques
- Ces signaux électriques sont ensuite convertis en une succession de 0 et de 1.
- Cette succession de 0 et de 1 est prise en charge par le processeur qui, en coopération avec un mémoire vive de stockage, traite cette information numérique suivant des programmes donnant les instructions à exécuter selon une méthode imposée : l'algorithme.
- Une fois ces calculs effectués, le processeur renvoie un résultat sous forme d'une succession aussi de 0 et de 1.
- Cette suite de nombres est reconvertie en signal électrique, signal qui reçu par l'écran ou les haut-parleurs conduit à un changement d'image ou de son.



LES PROCESSEURS

Le Cerveau de Votre Ordinateur

Découvrez tout sur les processeurs, les marques célèbres
et leurs différences clés



Qu'est-ce qu'un Processeur ?

Le **Processeur (CPU – Central Processing Unit)** est le composant principal d'un ordinateur qui effectue les calculs et exécute les instructions. C'est le "cerveau" qui prend toutes les décisions informatiques et rend les calculs possibles.



Exécution des Instructions

Le CPU traite et exécute toutes les instructions des programmes et applications que vous utilisez quotidiennement.



Calculs Mathématiques

Effectue des millions d'opérations mathématiques et logiques par seconde pour faire fonctionner vos logiciels.



Gestion des Données

Coordonne le flux de données entre tous les composants de l'ordinateur (RAM, disque dur, carte graphique).



Multitâche

Permet d'exécuter plusieurs programmes simultanément en gérant efficacement les ressources système.

Les Marques Principales

Découvrez les fabricants leaders qui dominent le marché des processeurs



Intel

Fondation : 1968

Séries : Core i3/i5/i7/i9,
Xeon

Marchés : Bureau,
Portable, Serveur

Point Fort : Performance
monocœur
exceptionnelle



AMD

Fondation : 1969

Séries : Ryzen,
Threadripper, EPYC

Marchés : Bureau,
Portable, Serveur

Point Fort : Meilleur
rapport qualité-prix



Apple Silicon

Lancement : 2020

Séries : M1, M2, M3, M4

Marchés : Mac, iPad

Point Fort : Efficacité
énergétique
révolutionnaire



ARM

Fondation : 1990

Séries : Cortex, Neoverse

Marchés : Mobile,
Tablettes, IoT

Point Fort : Architecture
mobile dominante

Différences Techniques Clés



Architecture

x86-64 (Intel, AMD) : Architecture traditionnelle des PC depuis des décennies

ARM (Apple Silicon, Qualcomm) : Architecture mobile, plus efficace énergétiquement



Processus de Fabrication

Intel : 10nm – 7nm

AMD : 7nm – 5nm

Apple : 3nm (M4)

Plus le nombre est petit, plus le processeur est efficace et puissant



Nombre de Cœurs

- ▶ **Grand Public** : 4 à 16 cœurs
- ▶ **Haut de Gamme** : Jusqu'à 64 cœurs (Threadripper)
- ▶ **Serveurs** : 128+ cœurs disponibles
- ▶ **Architecture Hybride** : Cœurs performance + efficacité



Graphiques Intégrés

- ▶ **Apple Silicon** : GPU exceptionnel intégré
- ▶ **Intel Iris Xe** : Amélioration significative
- ▶ **AMD Radeon Graphics** : Excellent pour le gaming léger
- ▶ Permet de se passer de carte graphique dédiée pour beaucoup d'usages

Core i3/i5/i7/i9 - Gammes de Produits Intel

Ce sont des **noms de gammes** qui indiquent le niveau de performance global :

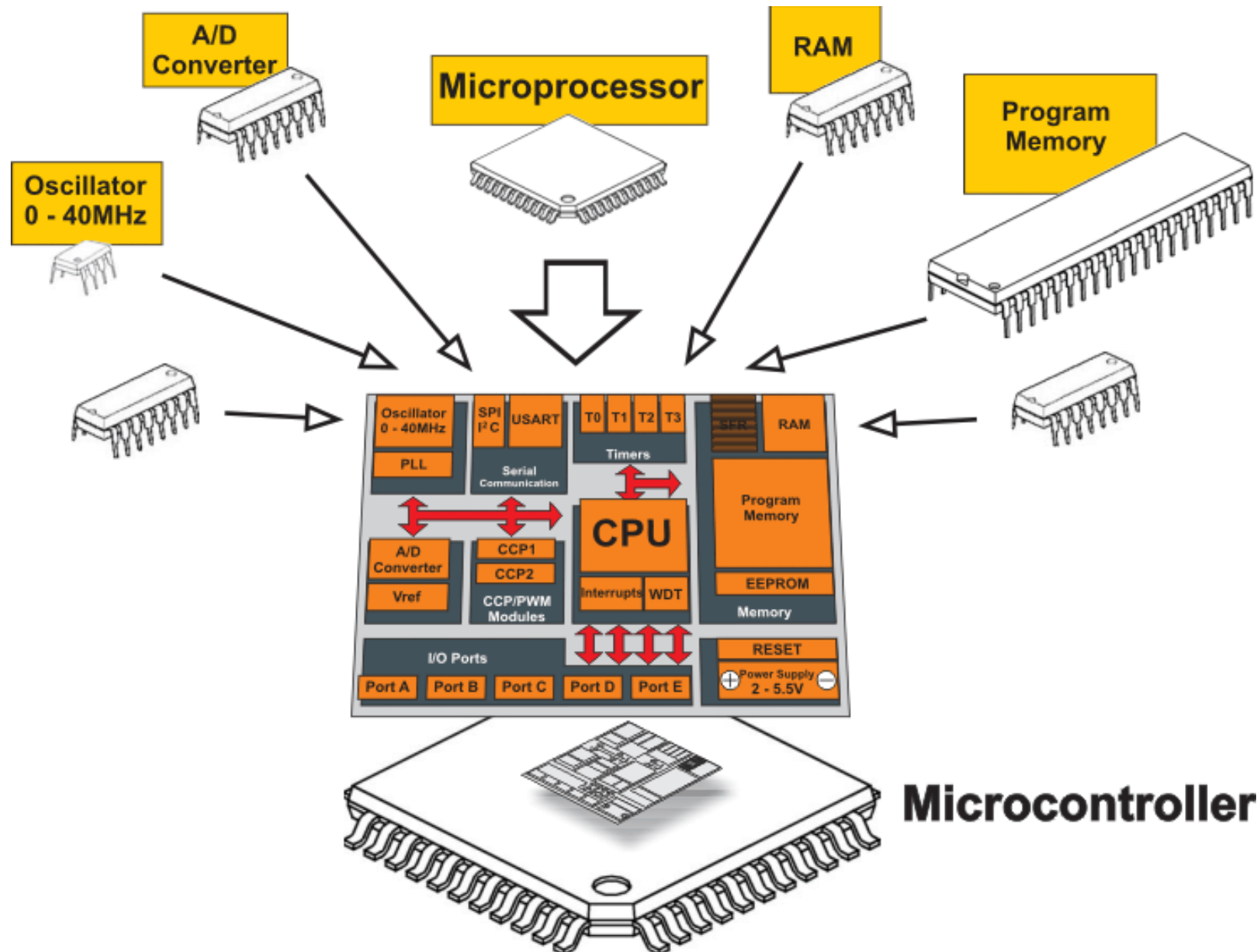
Gamme	Niveau	Nombre de Cœurs (Exemples)	Usage Typique
Core i3	Entrée de gamme	2-4 cœurs	Bureautique, navigation web, multimédia basique
Core i5	Milieu de gamme	4-6 cœurs	Multitâche, jeux, création de contenu amateur
Core i7	Haut de gamme	6-8 cœurs	Gaming haute performance, montage vidéo, rendu 3D
Core i9	Très haut de gamme	8-24 cœurs	Workstation professionnelle, streaming + gaming, calcul intensif

Points Importants à Retenir

- **Le nombre de cœurs varie selon la génération** : Un Core i5 de 10ème génération peut avoir un nombre de cœurs différent d'un Core i5 de 13ème génération
- **i3/i5/i7 ≠ nombre de cœurs** : Un Core i7 ne signifie PAS 7 cœurs ! Il peut en avoir 4, 6, 8 ou plus selon le modèle
- **Autres différences** : Les gammes i3/i5/i7 diffèrent aussi par la fréquence, le cache, l'hyperthreading, et d'autres technologies
- **AMD équivalent** : AMD a des gammes similaires (Ryzen 3, 5, 7, 9) avec la même logique

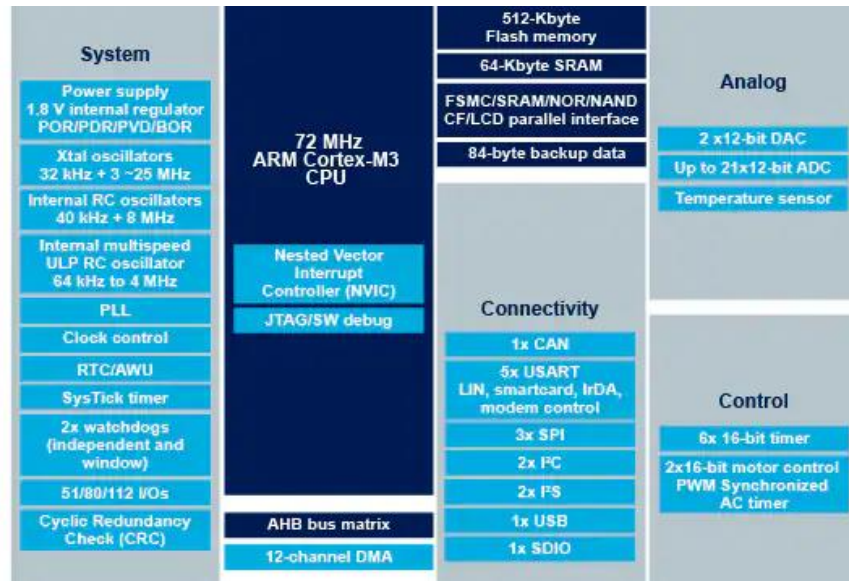
Processeur	Nombre de Cœurs	Utilisation Typique
Dual-Core	2 cœurs	Bureautique basique
Intel i3	2 à 6 cœurs	Navigation web, bureautique
Intel i5	4 à 14 cœurs	Gaming, multitâche, usage quotidien
Intel i7	6 à 20 cœurs	Gaming haut de gamme, création contenu
Intel i9	8 à 24 cœurs	Workstation, rendu 3D, streaming pro

Microcontrôleurs vs Microprocesseur



Microcontrôleurs vs Microprocesseur

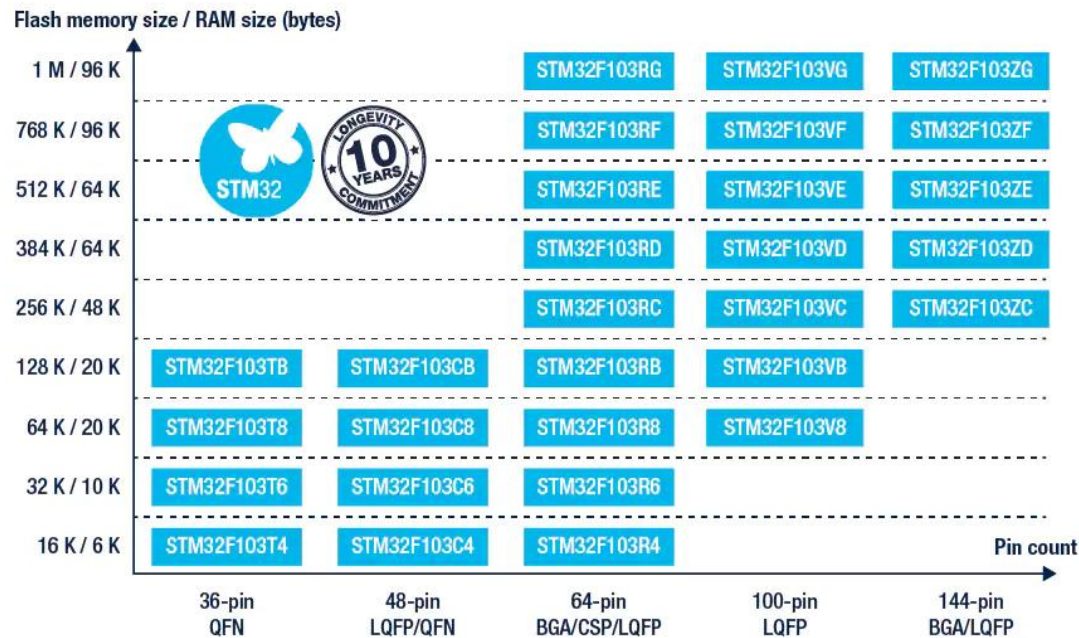
Un microcontrôleur est un circuit à base de microprocesseur qui contient (dans le même boîtier):



- Une mémoire pour les données.
- Une mémoire pour le programme.
- Des ports d'entrées/sorties
- D'autres périphériques comme: **des temporisateurs, des convertisseurs A<=>N, des interfaces de communications séries, parallèle, Ethernet, ...**

Il existe **différents types** de microcontrôleurs fabriqués par **divers fabricants**. Ce cours traite seulement les microcontrôleurs **STM32** de chez **STMicroelectronics**

Extrait de www.st.com



Deux types de processeurs

Il existe deux grandes familles au niveau des jeux d'instructions

- **CISC** : *Complex Instruction Set Computer*

Grand nombre d'instructions,
Type de processeur le plus répandu

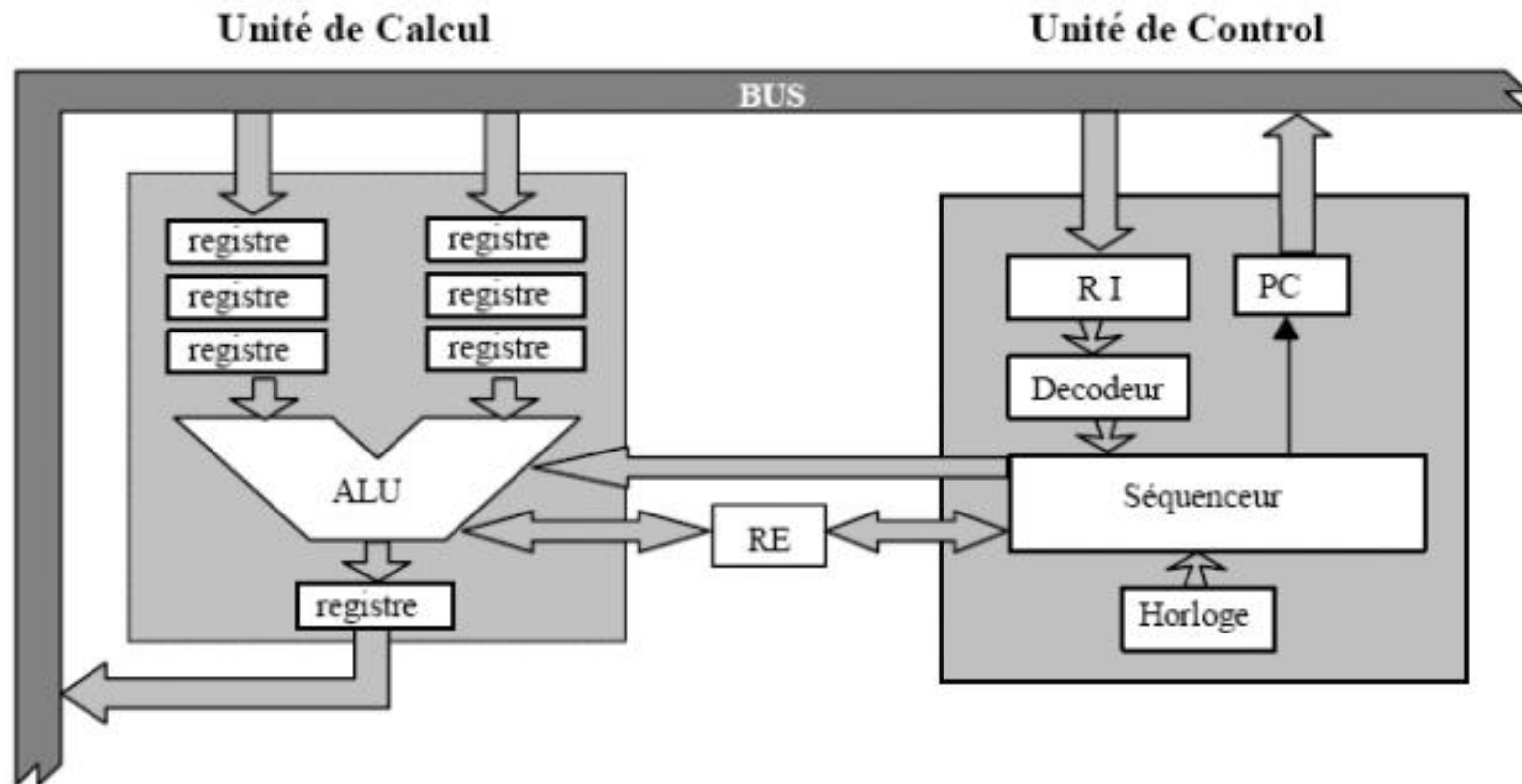
- **RISC** : *Reduced Instruction Set Computer*

Nombre d'instructions réduit
(sélection des instructions pour une exécution plus rapide)
Décodage des instructions plus rapide

Architecture interne d'un Microprocesseur

Le microprocesseur c'est puce électronique programmable

Les différents constituants d'un microprocesseur (μp) peuvent être regroupés dans deux blocs principaux, l'unité de calcul et l'unité de contrôle.



RI : registre d'instruction

PC : compteur de programme

RE : registre d'état

Fig. 1.1 : Architecture simplifiée d'un processeur

En mots simples un **microprocesseur**, c'est un CPU dans un circuit intégré

Architecture interne d'un Microprocesseur

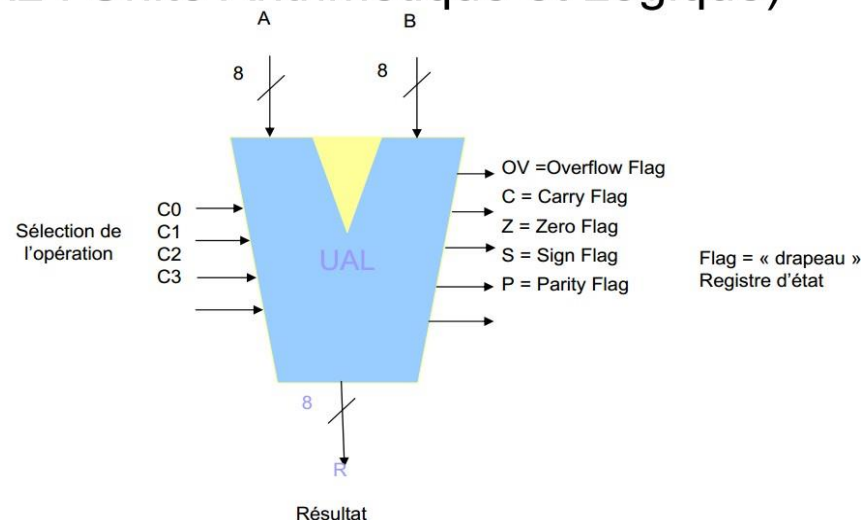
L'unité de calcul

Elle est constituée de l'Unité Arithmétique et logique UAL et d'un certain nombre de registres

☞ **ALU** : Unité arithmétique et logique

☞ **Les registres** : Un registre est une zone mémoire à l'intérieur du microprocesseur de faible taille, qui permet de mémoriser des mots-mémoires ou des adresses d'une façon temporaire lors de l'exécution des instructions

UAL : Unité Arithmétique et Logique)



- ALU ou UAL: regroupe l'ensemble des traitements pouvant être appliqués à des données

Architecture interne d'un Microprocesseur

➤ L'unité de contrôle

C'est l'unité de contrôle qui supervise le déroulement de toutes les opérations au sein du μp .

Elle est constituée principalement de :

- ☞ **Horloge** : C'est l'horloge qui génère les signaux qui permettent le cadencement et la synchronisation de toutes les opérations.
- ☞ **Le compteur programme PC** : (*Program Counter*) contient l'adresse de la case mémoire où est stockée la prochaine instruction à charger. Au début de l'exécution d'un programme, le PC est initialisé à l'adresse mémoire où est stockée la première instruction du programme. Le compteur programme est incrémenté chaque fois qu'une instruction est chargée dans le μp .
- ☞ **Le registre d'instruction RI** : C'est là où le μp stocke l'instruction en cours d'exécution.
- ☞ **Le décodeur** : C'est lui qui va "décoder" l'instruction contenue dans **RI** et générer les signaux logiques correspondants et les communiquer au séquenceur.
- ☞ **Le séquenceur** : Il gère le séquençement des opérations et génère :
 - les signaux du bus de commande (RD, WR, etc.),
 - les signaux internes aux μP (gestion des registres, de l'A.L.U., aiguillages des bus internes, etc.).
- ☞ **Le registre d'état** : Le registre d'état est formé de plusieurs bits appelés drapeaux ou indicateurs (Flags) qui sont positionnés par l'ALU après chaque opération. On dispose d'un jeu d'instructions conditionnées par l'état de différents drapeaux. Par exemple l'indicateur **Z** indique, quand il est positionné, que le résultat de l'opération est égal à Zéro. L'indicateur **C** indique que l'opération a généré une retenue. Le bit **N** indique que le résultat est négatif ...

Architecture interne d'un Microprocesseur

- Les différents bus d'un système micro-programmés

*« Un bus est un jeu de lignes partagées pour l'échange de mots numériques. »
(Traité de l'électronique, Paul Horowitz & Winfield Hill)*

Définition : Un bus permet de faire transiter (liaison série/parallèle) des informations codées en binaire entre deux points. Typiquement les informations sont regroupés en mots : octet (8 bits), word (16 bits) ou double word (32 bits).

Caractéristiques d'un bus:

- nombres de lignes,
- fréquence de transfert.

Architecture interne d'un Microprocesseur

Il existe 3 Types de bus :

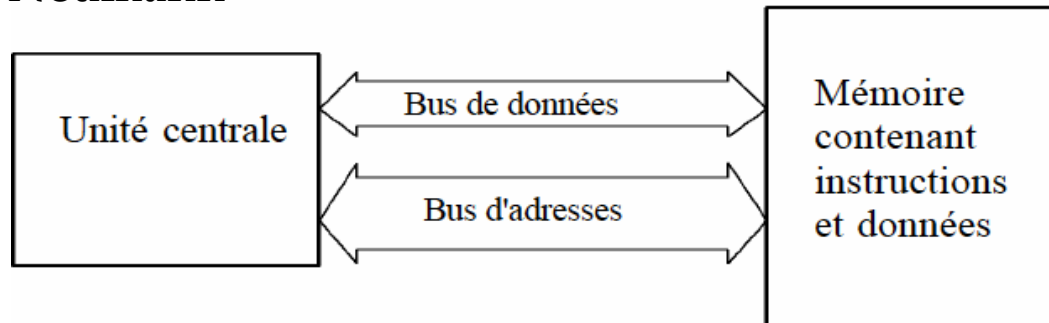
- **Bus de données** : permet de transférer entre composants des données,
ex. : résultat d'une opération, valeur d'une variable, etc.
- **Bus d'adresses** : permet de transférer entre composants des adresses,
ex. : adresse d'une case mémoire, etc.
- **Bus de contrôle** : permet l'échange entre les composants d'informations de contrôle [bus rarement représenté sur les schémas].
ex. : périphérique prêt/occupé, erreur/exécution réussie, etc.

Définition : Une **adresse** est un nombre binaire qui indique un emplacement dans une zone mémoire

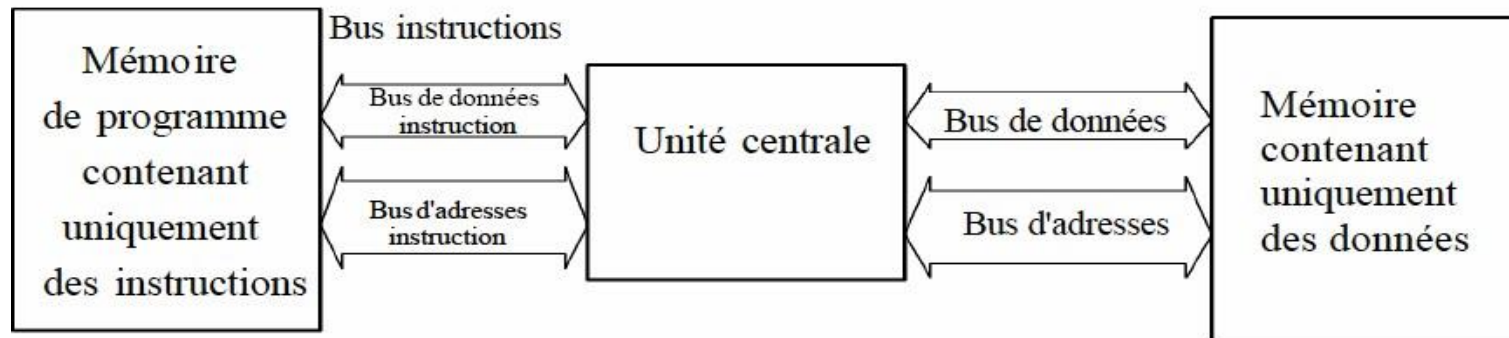
Structures des systèmes micro-programmés

Pour l'organisation des différentes unités, il existe deux architectures :

- **Structure de Von Neumann**



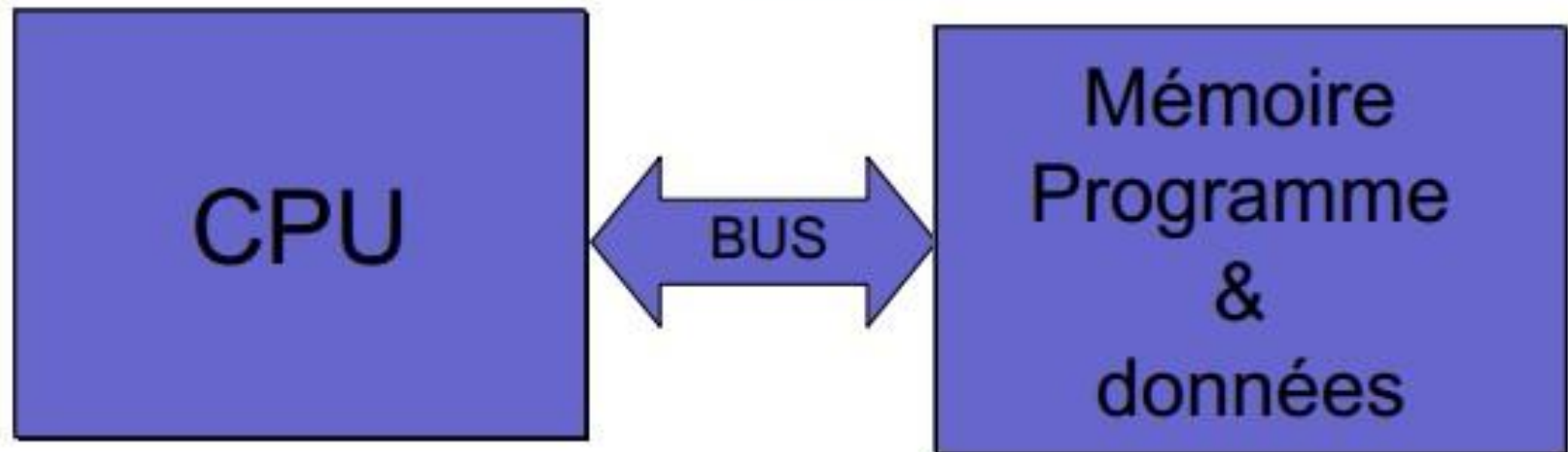
- **Structure de Harvard**



La différence se situe au niveau de la séparation ou non des mémoires programmes et données. La structure de Harvard permet de transférer données et instruction simultanément, ce qui permet un gain de performances.

L'architecture

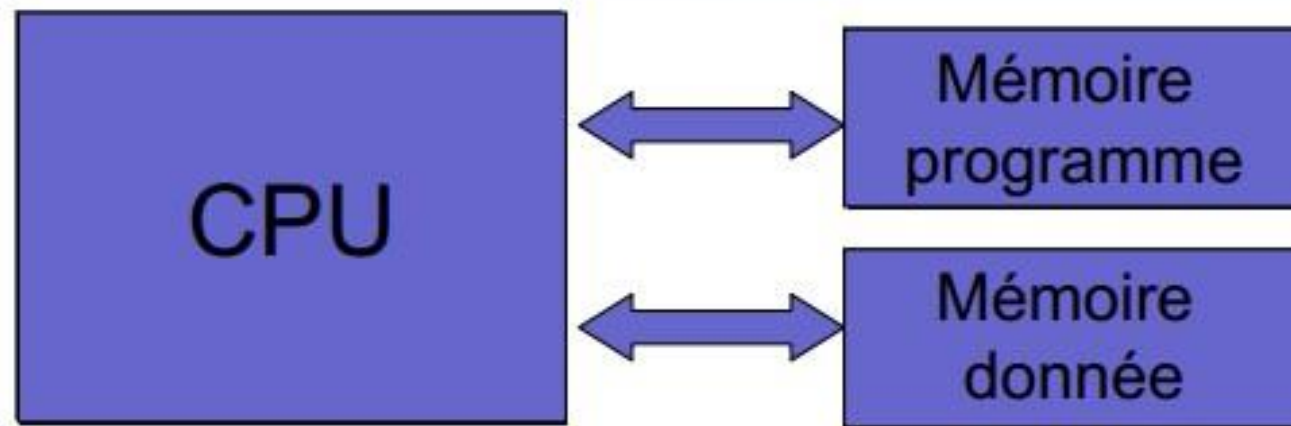
Von Neuman



- Un seul chemin d'accès à la mémoire
 - Un bus de données (programme et données),
 - Un bus d'adresse (programme et données)
- Architecture des processeurs d'usage général
- Goulot d'étranglement pour l'accès à la mémoire

L'architecture

Harvard

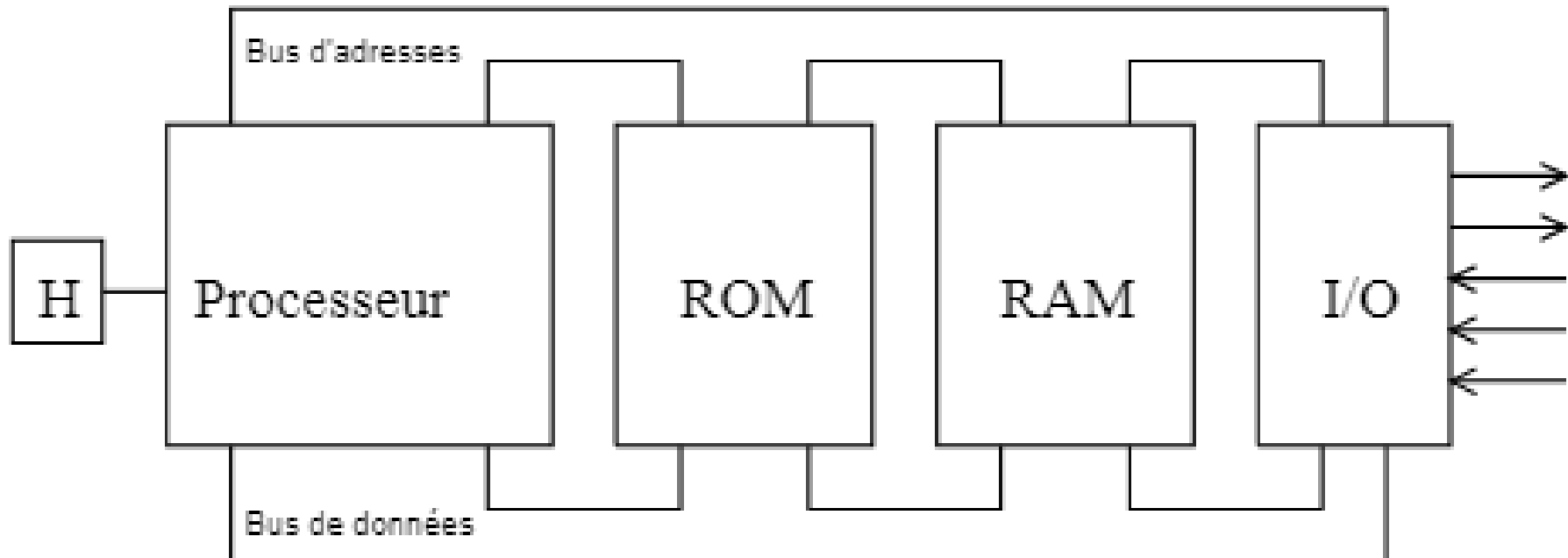


- Séparation des mémoires programme et données
 - Un bus de données programme,
 - Un bus de données pour les données,
 - Un bus d'adresse programme,
 - Un bus d'adresse pour les données.
- Meilleure utilisation du CPU :
 - Chargement du programme et des données en parallèle

Microcontrôleur

Un microcontrôleur est système informatique complet à l'intérieur d'un circuit intégré.

- Un **processeur**, qui exécute les instructions. Il est cadencé par une horloge (H).
- Une **mémoire morte** (ROM : *Read Only Memory*) qui contient des instructions. Son contenu est permanent : il reste intact lorsque le système n'est plus alimenté.
- Une **mémoire vive** (RAM : *Random Access Memory*, qui permet de stocker des données. Son contenu est perdu lorsque le courant est coupé.
- Des circuits d'**entrée-sortie** (I/O : *Input/Output*). Ce sont les circuits qui permettent l'interaction avec l'extérieur : clavier, souris, écran, mémoire externe, imprimante, etc.
- Des lignes électriques, appelées *bus* relient ces éléments entre eux. Voici un schéma-bloc d'un système informatique



Les différents systèmes programmables

- Les circuits spécialisés ou ASIC (Application Specific Integrated Circuit) :

Les circuits spécialisés sont des circuits spécialisés dès leur conception pour une application donnée.

Exemples : DSP (*Digital Signal Processing*), co-processeur arithmétique, processeur 3-D, contrôleur de bus, ...



Source : Texas Instruments



Source : NVidia

Avantages :

- Très rapide
- Consommation moindre
- Optimisé pour une application

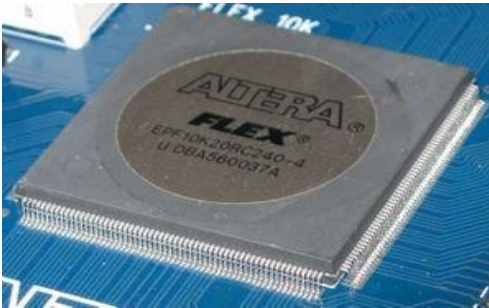
Inconvénients :

- Faible modularité
- Possibilité d'évolution limitée
- Coût

Les différents systèmes programmables

Les systèmes en logique programmée et/ou en logique programmable sont connus sous la désignation de PLD (*programmable logic device, circuit logique programmable*)

- **FPGA** (*field-programmable gate array, réseau de portes programmables in-situ*),
- **PAL** (*programmable array logic, réseau logique programmable*),
- ...



Source : Altera



Source : Altera

« Un circuit logique programmable, ou réseau logique programmable, est un circuit intégré logique qui peut être reprogrammé après sa fabrication. Il est composé de nombreuses cellules logiques élémentaires pouvant être librement assemblé. » (Wikipédia)

Avantages :

- Forte modularité
- Rapidité

Inconvénients :

- Mise en oeuvre plus complexe
- Coûts de développement élevé

Les différents systèmes programmables

- **Les systèmes micro-programmés :**

Les micro-contrôleurs sont typiquement des systèmes micro-programmés.



Micro-contrôleur Microchip
PIC16F690 en boîtier DIL20

Un **micro-contrôleur** est un :

« Circuit intégré comprenant essentiellement un microprocesseur, ses mémoires, et des éléments personnalisés selon l'application. »

Un micro-contrôleur contient un microprocesseur.

Avantages :

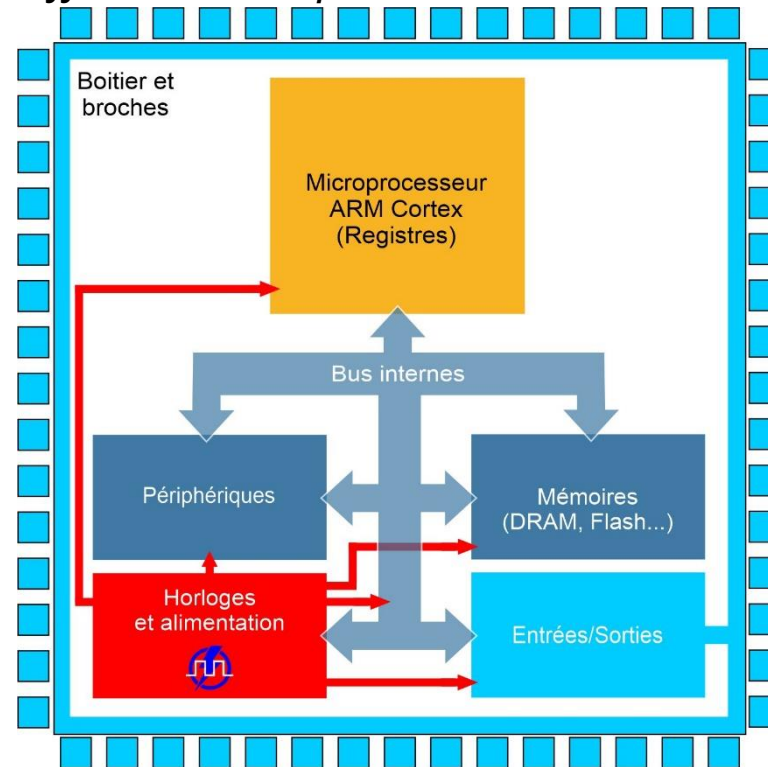
- Mise en oeuvre simple
- Coûts de développement réduits

Inconvénients :

- Plus lent
- Utilisation sous optimale

Architecture des microcontrôleurs STM32

- Un microcontrôleur est avant tout une puce électronique c'est-à-dire un assemblage de composants électroniques ultra-miniaturisés, de minuscules *transistors à effet de champ (MOS)* interconnectés selon des schémas très élaborés.



le schéma de principe d'un microcontrôleur

Les microcontrôleurs STM32 développés par la société STMicroelectronics¹ combinent plusieurs caractéristiques telles que la haute performance, le temps réel, la basse tension et la faible consommation, la diversité et sophistication des périphériques, un faible coût, une intégration complète et un développement facile.

Les microcontrôleurs STM32 sont construits autour d'un processeur ARM Cortex-M développé par la société ARM².

Architecture des microcontrôleurs STM32

Un **microprocesseur**, en l'occurrence un modèle Cortex conçu par la société ARM intégrant une unité de calcul généraliste.

est un calculateur très rapide qui opère sur des nombres écrits en notation binaire.

Les mémoires

Le microprocesseur a donc besoin de *mémoire* pour enregistrer les instructions des programmes qu'il exécute mais aussi les données qu'il manipule et les résultats de ses opérations.

registres : des mémoires très rapides, sont utilisés par le microprocesseur pour calculer et manipuler instructions et données pendant l'exécution des programmes,

RAM: mémoire vive.

les Flash et les EEPROM plus spacieuses et plus lentes, qui enregistrent les programmes et beaucoup de données. Elles tiennent pour le microcontrôleur le même rôle que le disque dur pour un ordinateur.

Les bus internes (et externes): Les bus sont constitués de fils conducteurs sur lesquels transitent les données et instructions des programmes.

Les entrées-sorties : permet au microcontrôleur d'interagir avec le monde extérieur

Les périphériques : On désigne par périphériques des circuits intégrés au microcontrôleur qui réalisent des fonctions parfois très spécialisées requises par les applications (conversion de signaux, pilotes d'écrans tactiles, communication avec des capteurs...).

exemples : CAN/CNA, UART, I2C, Timer, Watchdog...

Les horloges et les alimentations : On doit donc disposer d'un signal périodique et très stable pour synchroniser l'activité de tous les composants intégrés au microcontrôleur.

La très grande famille des ARMs



• *Historique*

- Historiquement, ARM: **A**corn **R**ISC **M**achine
- Acorn est une société anglaise (basée à Cambridge) qui, à la fin des années 70 a conçu et fabriqué un processeur 8 bits, le 6502, sur lequel la société base un ensemble de micro-ordinateurs.
- Le plus emblématique est le « **BBC micro** », vendu à 1,5 millions d'exemplaires (85% des ventes d'ordinateurs aux écoles anglaises).
- Au milieu des années 80, Acorn se lance dans le développement d'une nouvelle génération de processeur basée sur la technologie RISC.
- En avril 1985, après seulement 18 mois de travaux, l'ARM 1 est le 1er (co)processeur RISC à être commercialisé
- En 1987, le premier ordinateur à base de cœur ARM est l'Acorn Archimedes (ARM 2 puis ARM 3)
- Là où Apple, Atari et Commodore, avec un Motorola 68000 à 8 Mhz ne disposait que d'une puissance de 1 MIPS, Acorn disposait lui d'une puissance de 4,5 MIPS à même fréquence avec son processeur maison.
- En novembre 1990, face à l'intérêt d'Apple vis-à-vis de l'architecture des processeurs ARM, la joint venture ARM est créée entre Appel, Acorn et VLSI
- En 1994, le RISC PC (ARM610, ARM700 puis ARM 710 et 710a) succède à l'Acorn Archimedes (équipé du RISC OS).

Architecture des microcontrôleurs STM32



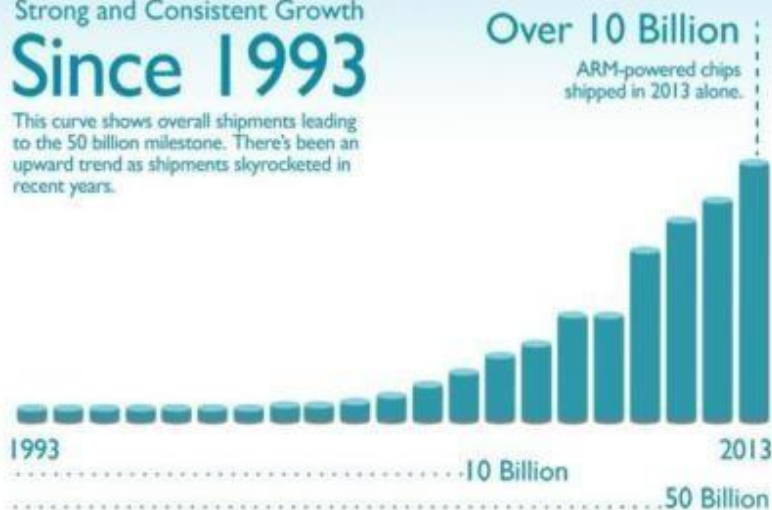
ARM
Everywhere

With More Than
50 BILLION

ARM powered chips shipped by our partners,
we're helping power more and more applications worldwide.

Strong and Consistent Growth
Since 1993

This curve shows overall shipments leading to the 50 billion milestone. There's been an upward trend as shipments skyrocketed in recent years.



Markets we're

POWERING

Where have 50 billion chips gone?
We break it down by market type below.



20% | Embedded

Applications including automotive, touch-screen controllers, industrial equipment, connectivity and smartcards



16% | Enterprise

Applications such as hard disk drives, and wireless/wireline networking infrastructure equipment



58% | Mobile

Devices including smartphones, mobile phones, tablets, e-readers and wearables



6% | Home

Consumer devices such as smart TVs, game consoles and home networking gateways



La très grande famille des ARMs

Quelques processeurs emblématiques à cœur(s) ARM

- Famille **AT91** d'Atmel (SAM9 / SAM7)
- Famille **LPC** de NXP
- Famille **OMAP** de Texas Instrument
- Famille **iMX** de Freescale
- Famille **STM32** de ST Microelectronics
- Famille **Tegra** de Nvidia
- Famille **Snapdragon** de Qualcomm
- Famille **AXX** de Allwinner
- Famille **Exynos** de Samsung
- Famille **AX** d'Apple
- ARM11 (xxx)
 - Mobiles Nokia, HTC, Samsung, ZTE, iPhone original et 3G, Nintendo 3DS, Radpberry Pi
- Cortex A7
 - Raspberry Pi 2
- Cortex A8
 - iPhone 3GS, HTC Desire, Pandara board, Motorola Droid, Samsung Galaxy S...
- Cortex A9
 - Samsung Galaxy SII, iPhone 4S, Pandaboard, NAS Synology DS215j ...
- Cortex M3 : Arduino Due, STM32



La très grande famille des ARMs



Huge Range of Applications

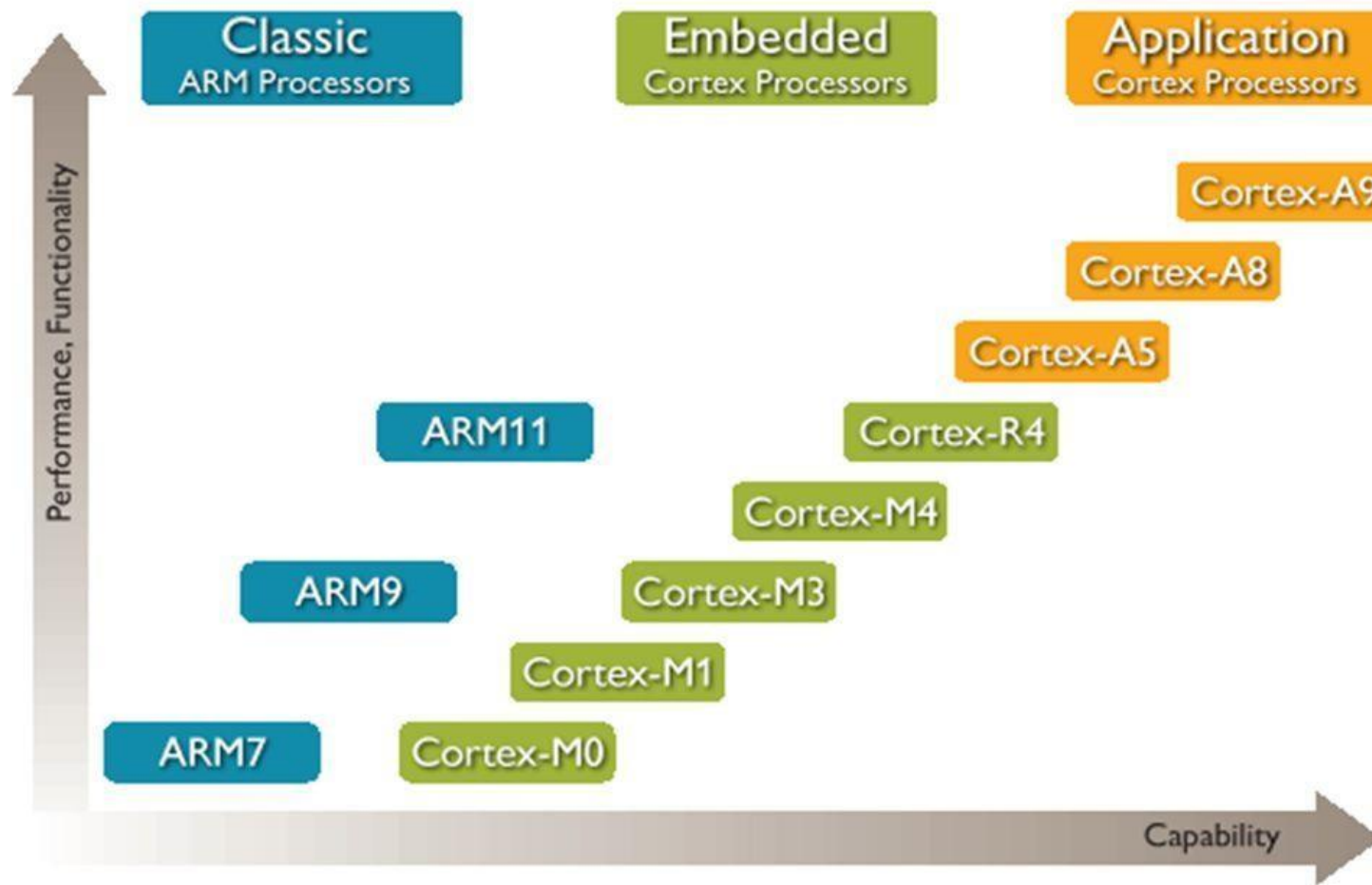


ARM University Program

The Architecture for the Digital World® **ARM**

La très grande famille des ARM^s

La famille ARM



Processeur ARM-Cortex

- ARM est une société britannique spécialisée dans le développement de processeurs d'architecture 32 bits et d'architecture 64 bits de type [RISC](#).
- La société anglaise [ARM](#) est à l'origine de l'architecture de microprocesseurs [Cortex M](#) spécialement conçue pour les microcontrôleurs.
- Une particularité des processeurs ARM est leur mode de vente. En effet, ARM Ltd. ne fabrique ni ne vend ses processeurs sous forme de circuits intégrés. Cette société vend les licences de ses processeurs à d'autres fabricants, qui pourront les intégrer dans leurs puces. Aujourd'hui, la plupart des grands fondeurs de puces ont acheté des licences à ARM et proposent des systèmes sur puces intégrant des microprocesseurs Cortex. *Wikipedia*

Cortex - A

Highest performance

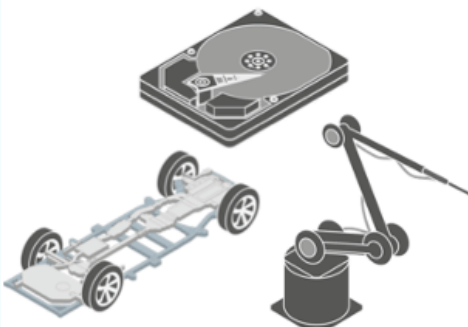
Optimised for
rich operating systems



Cortex - R

Fast response

Optimised for
high performance,
hard real-time applications



Cortex - M

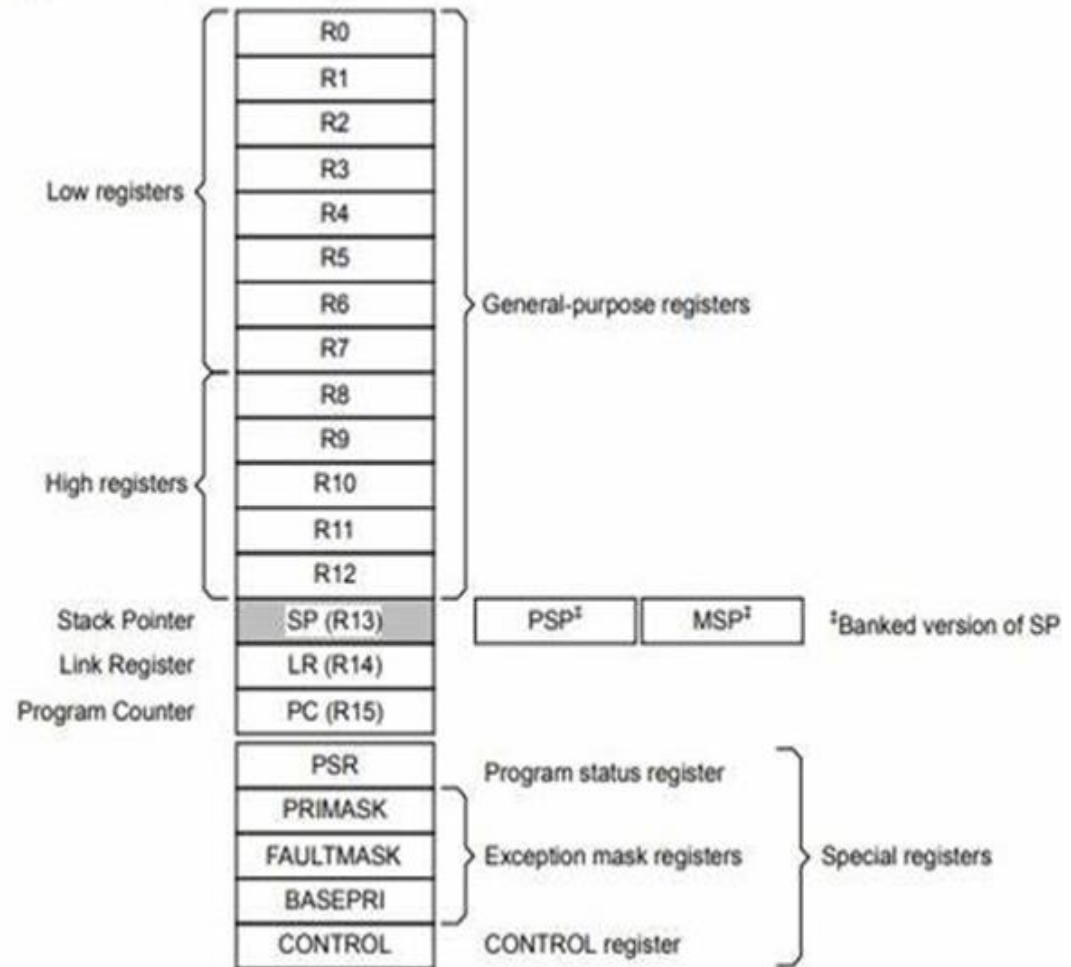
Smallest/lowest power

Optimised for
discrete processing and
microcontrollers



Processeur ARM-Cortex

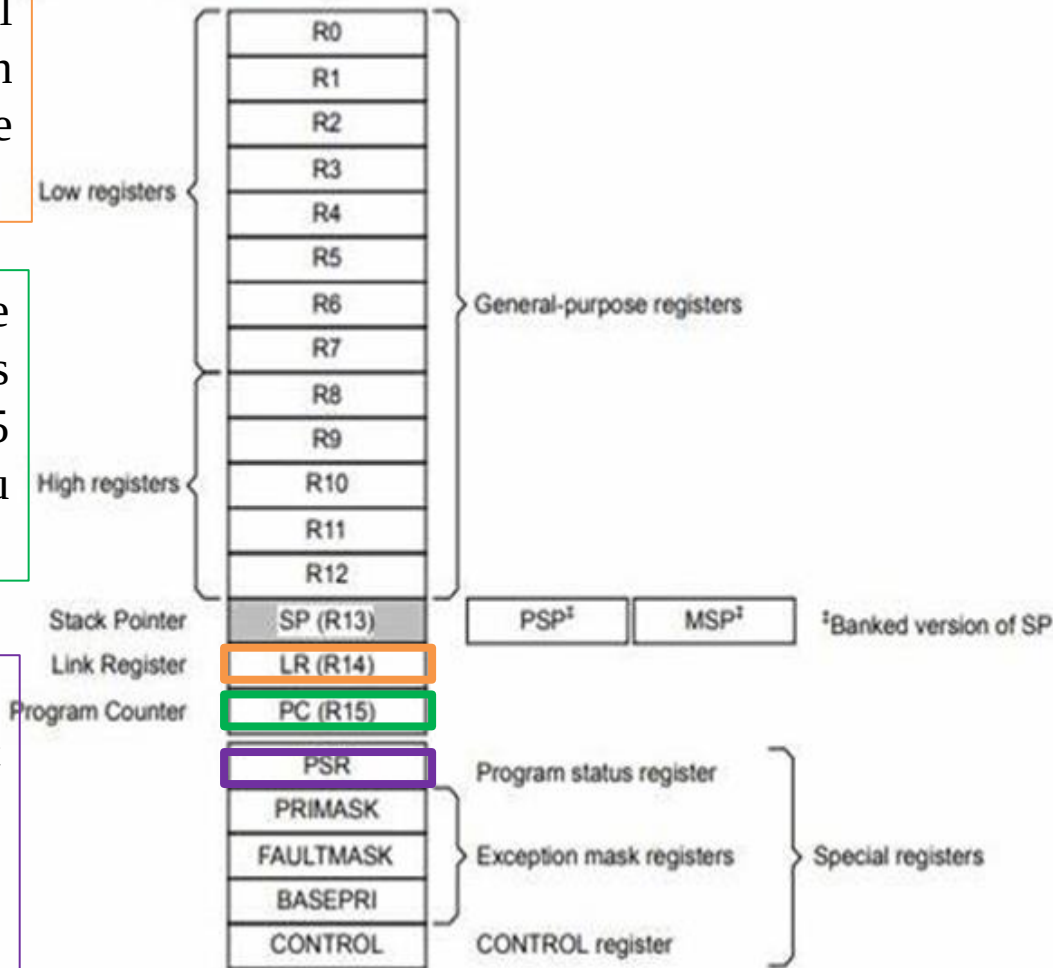
- Les Registres Bas (r0 à r7) sont des registres tout usage disponibles pour toutes les instructions, quelles soient encodées sur 16 bits ou sur 32 bits.
- Les registres hauts sont aussi des registres tout usage disponibles uniquement pour les instructions 32 bits.
- *Il faut 3 bits pour désigner un registre parmi les registres bas. Il faut 4 bits pour désigner un registre parmi les 16 registres de r0 à r15!*



Registre de base du processeur Cortex M3

Processeur ARM-Cortex

- Le registre de lien (RL) est automatiquement mis à jour lors de l'appel de fonction: il contient l'adresse de retour de la fonction (l'adresse de l'instruction à exécuter lorsque l'exécution de la fonction sera terminée).
- Le registre (PC) est le compteur de programme. Ce registre indique l'adresse des instructions à être exécuter. Changer R15 équivaut à faire un saut à travers le code du programme
- Le registre PSR(program status register, registre d'état du programme), indique l'état de l'exécution d'un programme, il peut indiquer la valeur du drapeaux de l'ALU, l'état de l'exécution ou numéro de l'interruption en cours.



Registre de base du processeur Cortex M3

Processeur ARM-Cortex

- Le registre PSR(program status register, registre d'état du programme)

Le PSR (*Program Status Register*, registre d'état du programme) est représenté à la figure 4.

Les quatre premiers bits sont des drapeaux de code condition, appelés N (*Negative*, négatif), Z (*Zero*, zéro), C (*Carry*, retenue) et V (*oVerflow*, dépassement) et positionnés suivant le résultat de l'instruction de traitement de l'opération. Le bit Q est utilisé par les instructions mathématiques pour indiquer qu'une variable a atteint sa valeur maximale ou minimale. Les champs 26-25 et 15-10 sont utilisés dans les branchements conditionnels et le champ 8-0 pour les interruptions.

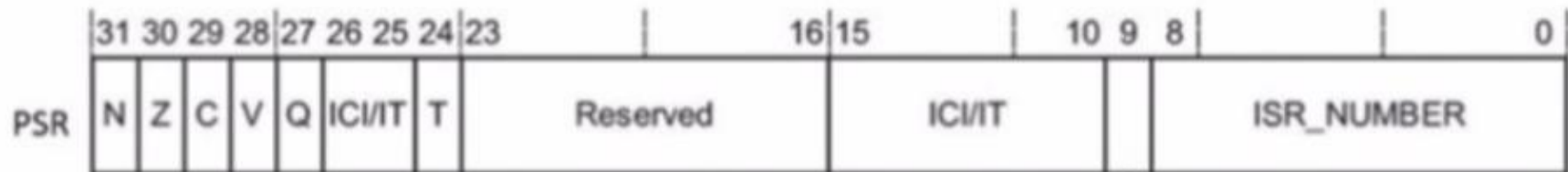
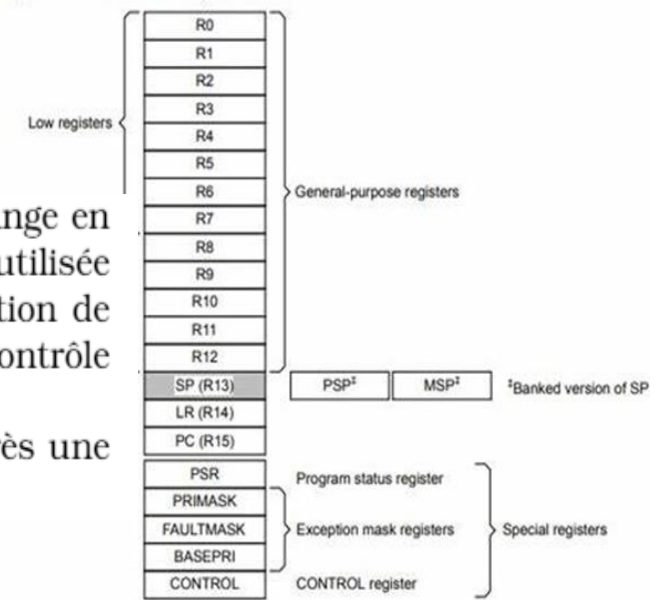


FIGURE 4 – Représentation du PSR, le registre d'état du programme.

Processeur ARM-Cortex

Le registre R13 est le SP (*Stack Pointer*, pointeur de pile). Le contenu du registre change en fonction du mode CPU actuel (privilegié ou non). Cette fonction est généralement utilisée par les systèmes d'exploitation en temps réel (RTOS) pour effectuer une commutation de contexte. Lorsque le Cortex-M se trouve en mode *Thread*, c'est le bit 1 du registre de contrôle (CONTROL) qui indique le pointeur de pile à utiliser :

- 0 pour MSP (*Main Stack Pointer*, pointeur de pile principal) : c'est le pointeur après une réinitialisation du Cortex-M ;
- 1 pour PSP (*Process Stack Pointer*, pointeur de pile de traitement).



PRIMASK (*PRIority mask register*, registre de masque de priorité), FAULTMASK (*FAULT MASK register*, registre de masque de faute) et BASEPRI (*BASE PRIority mask register*, registre de masque de base) sont utilisés pour le contrôle des exceptions/interruptions.

CONTROL (registre de contrôle) détermine la pile utilisée et le niveau de privilège pendant l'exécution d'un programme en mode *Thread*.

Non privilégié

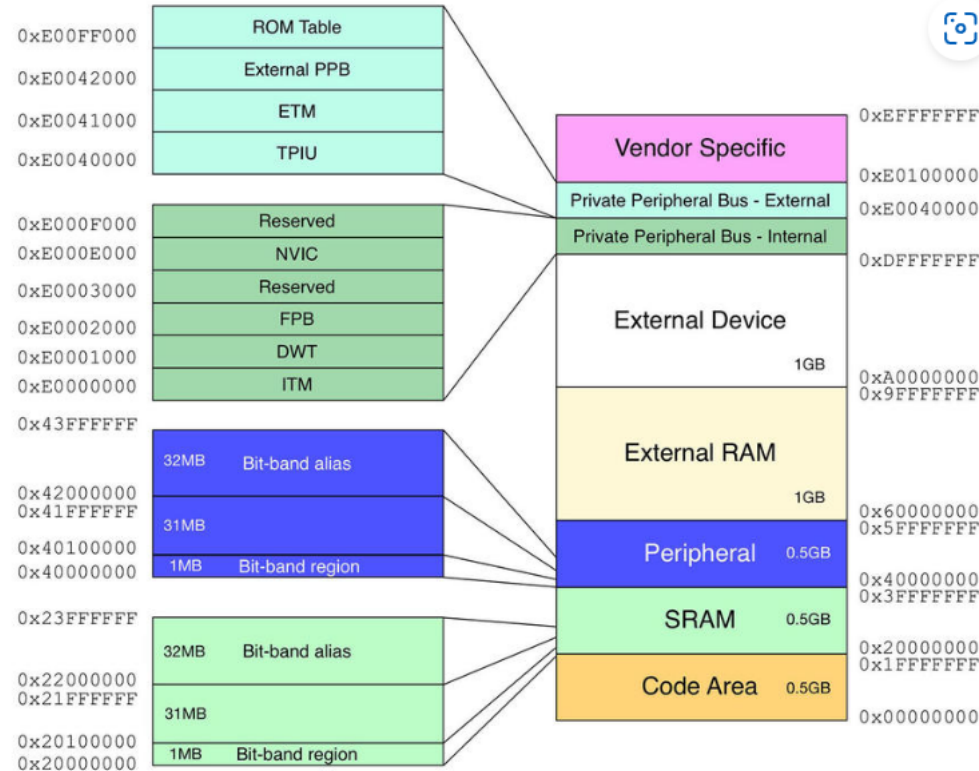
- A un accès limité aux instructions
- Impossible d'accéder à la minuterie système, ou au bloc de contrôle système.
- Peut avoir un accès restreint à la mémoire ou aux périphériques.

Privilegié (par défaut)

- Peut utiliser toutes les instructions et a accès à toutes les ressources.
- Peut écrire dans le registre CONTROL pour modifier le niveau de privilège pour l'exécution du logiciel

La mémoire du ARM-Cortex M3

- ARM définit un *espace d'adressage mémoire standardisé* commun à tous les cœurs Cortex-M, ce qui garantit la portabilité du code entre les différents fabricants de silicium. **L'espace d'adressage a une largeur de 4 Go** (en raison de la ligne d'adresse 32 bits), et il est organisé en plusieurs sous-régions avec des fonctionnalités logiques différentes.
- Par défaut, un mot de mémoire est de 8 bits.**



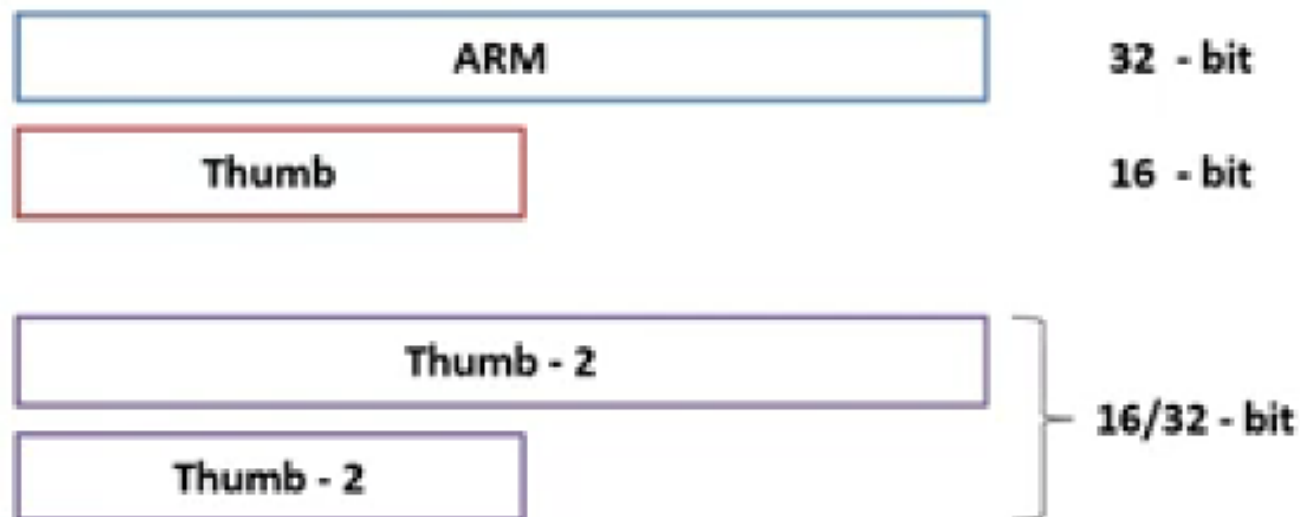
Correction de la carte mémoire pour les cœurs ARM

Jeux d'instruction

Les processeurs ARM Cortex-M fournissent un jeu d'instructions 32 bits, permettent non seulement un riche ensemble d'instructions, mais garantissent également les meilleures performances. Cependant, l'encombrement mémoire du micrologiciel a un coût plus élevé. Pour résoudre ces problèmes, ARM a introduit le jeu d'instructions *Thumb* 16 bits qui est étendu de manière transparente aux instructions ARM 32 bits complètes en temps réel, sans perte de performances. Par la suite, ARM a introduit le jeu d'instructions *Thumb-2*, qui est un mélange de jeux d'instructions 16 et 32 bits dans un état de fonctionnement.

2 jeux d'instructions sont supportés :

- Les instructions ont toutes 32 bits si le jeu d'instruction Thumb est actif.
- Les instructions ont 16 bits pour économiser de la mémoire au besoin



Microcontrôleurs STM32

STM32 est une large gamme de microcontrôleurs ARM Cortex-M divisés en neuf sous-familles. En interne, chaque microcontrôleur se compose du cœur du **processeur**, de la **RAM statique**, de la **mémoire flash**, de **l'interface de débogage** et de divers périphériques.

Voici les avantages de l'utilisation des microcontrôleurs STM32 :

- Les microcontrôleurs basés sur Cortex-M disposent d'une grande communauté, soutenue par une chaîne d'outils gratuite, et sont écrits dans de nombreux articles de connaissances partagées
- La compatibilité broche à broche pour la plupart des microcontrôleurs STM32 permet de modifier le microcontrôleur tout en conservant les affectations de broches
- Presque les broches sont tolérantes à **5 V**, ce qui signifie qu'elles peuvent s'interfacer avec d'autres appareils qui n'utilisent pas 3,3 V sans utiliser de décalage de niveau
- L'utilisation de microcontrôleurs STM32 avec des processeurs ARM et des RTOS pris en charge est bon marché.
- Le chargeur de démarrage intégré est livré avec une ROM interne qui permet de reprogrammer la mémoire flash interne à l'aide de certains périphériques de communication



STM32 MCUs

32-bit Arm® Cortex®-M



High
Performance

STM32F2

398 CoreMark
120 MHz Cortex-M3

STM32F4

608 CoreMark
180 MHz Cortex-M4

STM32F7

1082 CoreMark
216 MHz Cortex-M7

STM32H7

Up to 3224 CoreMark
Up to 550 MHz
Cortex-M7
240 MHz Cortex-M4



Mainstream

STM32G0

142 CoreMark
64 MHz Cortex-M0+

STM32G4

550 CoreMark
170 MHz Cortex-M4

STM32F0

106 CoreMark
48 MHz Cortex-M0

STM32F1

177 CoreMark
72 MHz Cortex-M3

STM32F3

245 CoreMark
72 MHz Cortex-M4

Optimized for
mixed-signal applications



Ultra-low-
power

STM32L0

75 CoreMark
32 MHz Cortex-M0+

STM32L1

93 CoreMark
32 MHz Cortex-M3

STM32L4

273 CoreMark
80 MHz Cortex-M4

STM32L4+

409 CoreMark
120 MHz Cortex-M4

STM32U5

651 CoreMark
160 MHz Cortex-M33

STM32L5

443 CoreMark
110 MHz Cortex-M33



Wireless

STM32WL

162 CoreMark
48 MHz Cortex-M4
48 MHz Cortex-M0+

STM32WB

216 CoreMark
64 MHz Cortex-M4
32 MHz Cortex-M0+

Cortex-M0+
Radio co-processor

La très grande famille des ARM

La famille ARM

Common core peripherals and architecture:

Communication peripherals: USART, SPI, I2C
Multiple general-purpose timers
Integrated reset and brown-out warning
Multiple DMA
2x watchdogs
Real-time clock
Integrated regulator PLL and clock circuit
External memory interface (FSMC)
Up to 3x 12-bit DAC
Up to 4x 12-bit ADC (Up to 5 MSPS)
Main oscillator and 32 kHz oscillator
Low-speed and high-speed internal RC oscillators
-40 to +85 °C and up to 105 °C operating temperature range
Low voltage 2.0 to 3.6 V or 1.65/1.7 to 3.6 V (depending on series)
Temperature sensor

+

STM32 F4 series - High performance with DSP (STM32F401/405/415/407/417/427/437/429/439)

180 MHz Cortex-M4 with DSP and FPU	Up to 256-Kbyte SRAM	Up to 2-Mbyte Flash	2x USB 2.0 OTG FS/HS	3-phase MC timer	2x CAN 2.0B	SDIO 2x I2S audio Camera IF	Ethernet IEEE 1588	Crypto TFT LCD + SDRAM
------------------------------------	----------------------	---------------------	----------------------	------------------	-------------	-----------------------------	--------------------	------------------------



STM32 F3 series - Mixed-signal with DSP (STM32F302/303/313/372/373/383)

72 MHz Cortex-M4 with DSP and FPU	Up to 48-Kbyte SRAM & DCM-SRAM	Up to 256-Kbyte Flash	USB 2.0 FS	2x 3-phase MC timer (144 MHz)	CAN 2.0B	Up to 7x comparator	3x 16-bit $\Sigma\Delta$ ADC	4x PGA
-----------------------------------	--------------------------------	-----------------------	------------	-------------------------------	----------	---------------------	------------------------------	--------



STM32 F2 series - High performance (STM32F205/215/207/217)

120 MHz Cortex-M3 CPU	Up to 128-Kbyte SRAM	Up to 1-Mbyte Flash	2x USB 2.0 OTG FS/HS	3-phase MC timer	2x CAN 2.0B	SDIO 2x I2S audio Camera IF	Ethernet IEEE 1588	Crypto
-----------------------	----------------------	---------------------	----------------------	------------------	-------------	-----------------------------	--------------------	--------



STM32 F1 series - Mainstream - 5 product lines (STM32F100/101/102/103 and 105/107)

Up to 72 MHz Cortex-M3 CPU	Up to 96-Kbyte SRAM	Up to 1-Mbyte Flash	USB 2.0 OTG FS	3-phase MC timer	Up to 2x CAN 2.0B	SDIO 2x I2S audio	Ethernet IEEE 1588	
----------------------------	---------------------	---------------------	----------------	------------------	-------------------	-------------------	--------------------	--



STM32 F0 series - Entry level (STM32F030/50/051)

48 MHz Cortex-M0 CPU	Up to 8-Kbyte SRAM	Up to 64-Kbyte Flash	3-phase MC timer	Comparator	CEC			
----------------------	--------------------	----------------------	------------------	------------	-----	--	--	--



STM32 L1 series - Ultra-low-power (STM32L100/151/152/162)

32 MHz Cortex-M3 CPU	Up to 48-Kbyte SRAM	Up to 384-Kbyte Flash	USB FS device	Up to 12-Kbyte EEPROM	LCD 8x40 4x44	Comparator	BOR MSI VScal	AES 128-bit
----------------------	---------------------	-----------------------	---------------	-----------------------	---------------	------------	---------------	-------------

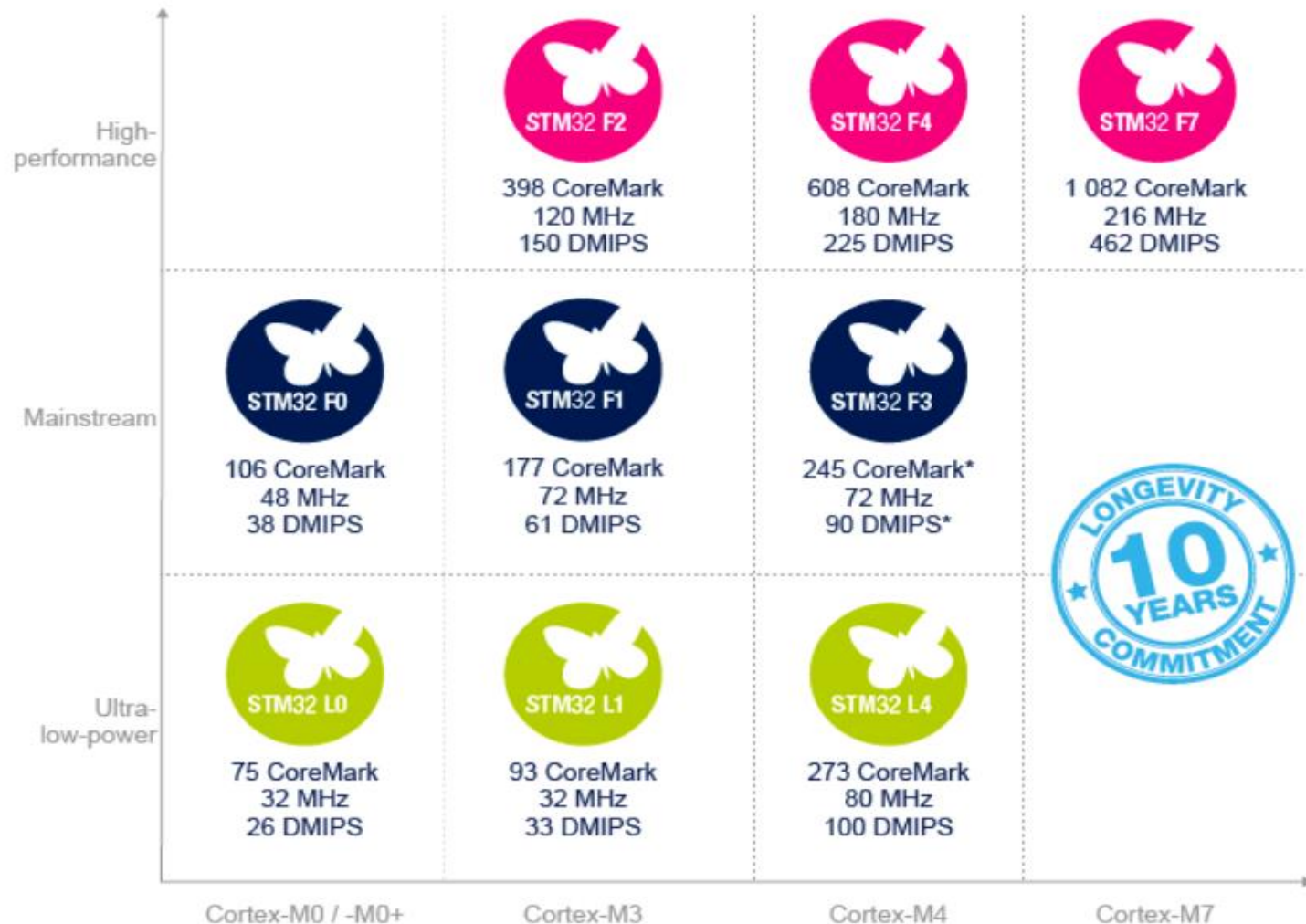


STM32 W series - Wireless (STM32W108)

24 MHz Cortex-M3 CPU	Up to 16-Kbyte SRAM	Up to 256-Kbyte Flash	2.4 GHz IEEE 802.15.4 Transceiver	Lower MAC Digital baseband	AES 128-bit			
----------------------	---------------------	-----------------------	-----------------------------------	----------------------------	-------------	--	--	--

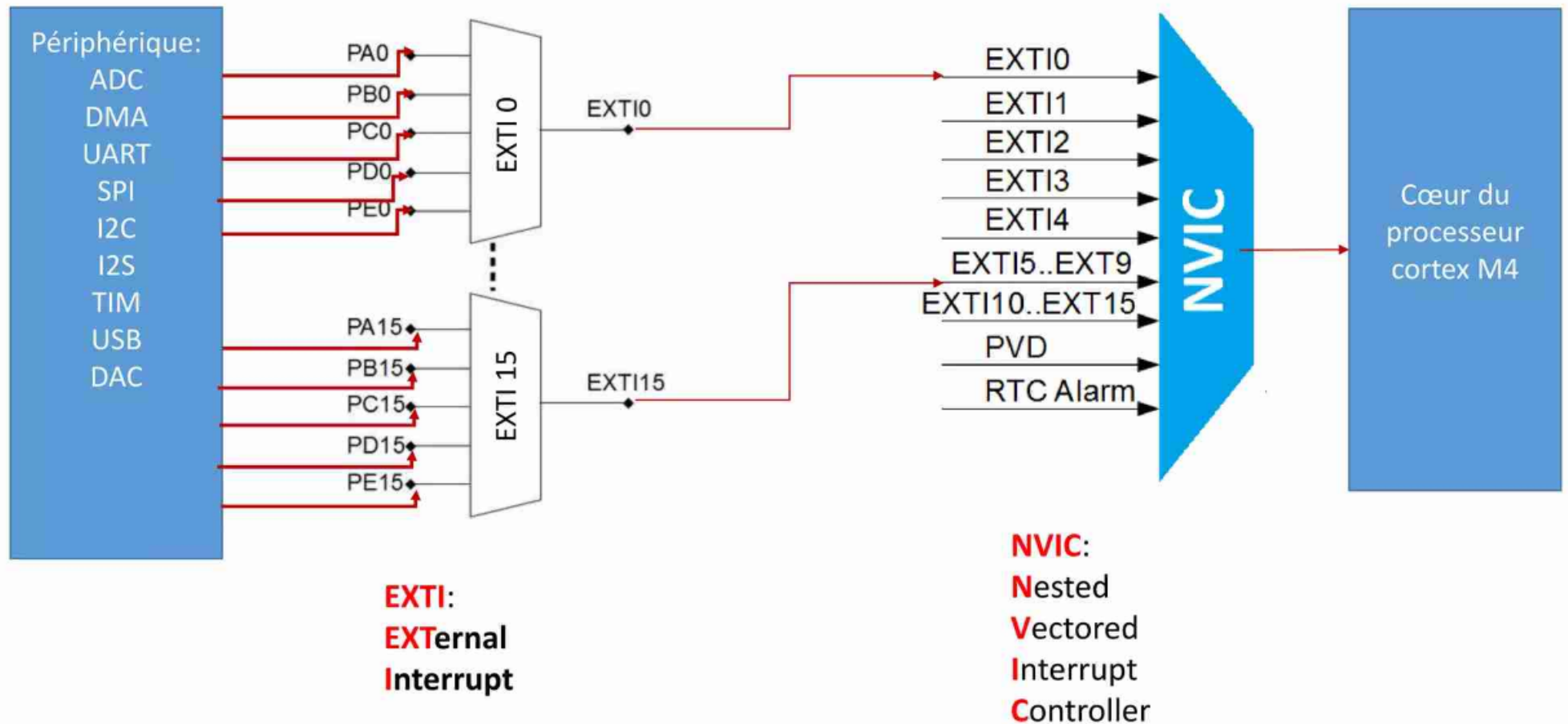


Série STM32

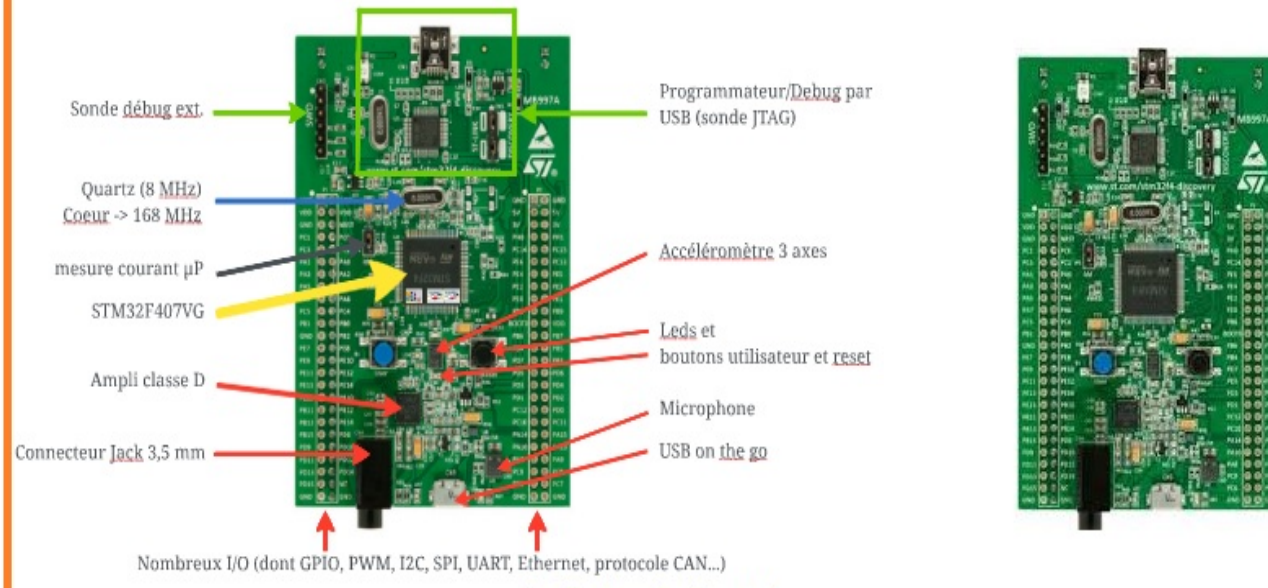


Comme le montre la figure 9, certaines séries de STM32 sont considérées comme des *Ultra Low Power* (très faible consommation), d'autres comme des *Mainstream* (pour des applications d'exigences moyennes) et d'autres enfin comme des *High Performance* (hautes performances). Néanmoins, toutes les séries ont la même architecture et un certain nombre de fonctions identiques, comme indiqué dans la colonne de gauche. Selon la série, la vitesse du microcontrôleur, la taille des mémoires associées et la nature des périphériques spécifiques associés diffèrent.

Contrôleur des interruptions



Introduction to STM32F4



Caractéristique du microcontrôleur stm32f407vg

STM32F407VG



Fabricant	STMicroelectronics
Largeur du bus de données	32 bit
Fréquence de l'horloge max	168 MHz
Taille de la mémoire du programme	1 MB
Taille de la RAM de données	192 KB
Cœur	ARM Cortex M4
Tension d'alimentation de fonctionnement	1.8 V to 3.6 V
Température de fonctionnement max	+ 85 C
Type de Ram de données	SRAM
Type d'interface	CAN, I2C, I2S, SPI, UART
Température de fonctionnement min	- 40 C
Nombre d'E/S	82 I/O
Type de la mémoire programme	Flash

STM32F4xx Block Diagram

Périphérique de communication:

■ Up to 15 communication interfaces

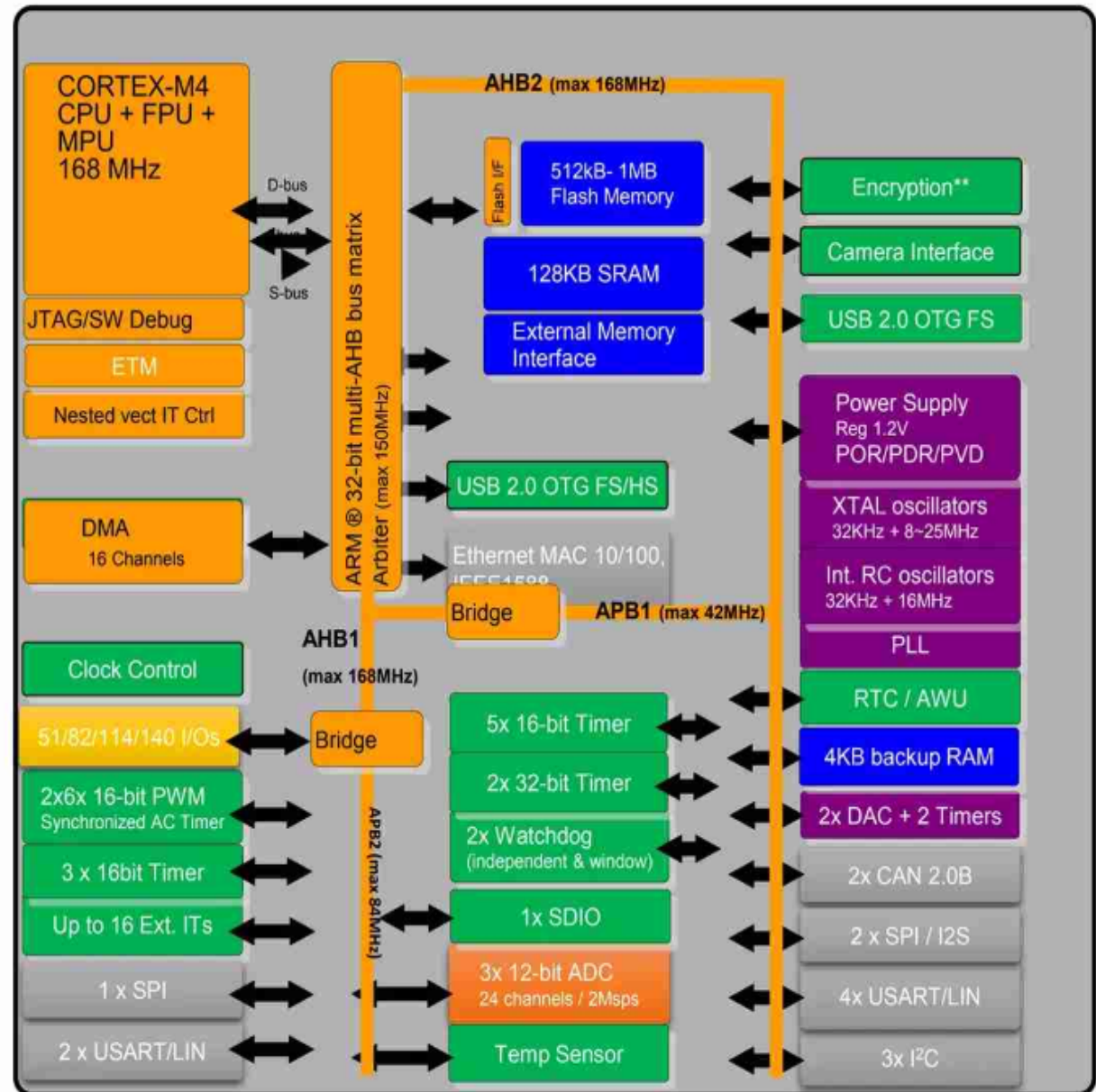
- Up to 3 × I2C interfaces
- Up to 4 USARTs/2 UARTs
- Up to 3 SPIs (37.5 Mbits/s),
- 2 × CAN interfaces (2.0B Active)
- SDIO interface

■ Advanced connectivity

- USB 2.0 full-speed device
- USB 2.0 high-speed/full-speed device

- 10/100 Ethernet MAC

- 8- to 14-bit parallel camera interface up to 54 Mbytes/s



Prise en main STM32CubeIDE

Objectif :

- Prise en main de la plateforme de développement de STM32CubeIDE
- apprendre à contrôler les broches GPIO de STM32 Blue Pill. À titre d'exemple, nous allons voir comment les configurer en tant que broches de sortie numérique à l'aide du pilote **GPIO HAL** et de l'IDE STM32Cube.

Prise en main STM32CubeIDE



Utilisation de la bibliothèque HAL (Hardware Abstraction Layer) avec l'interface GPIO

HAL GPIO fournit les fonctions permettant de contrôler, initialiser/désinitialiser les broches GPIO d'un μ C STM32

Initialization and de-initialization functions

This section contains the following APIs:

- `HAL_GPIO_Init()`
- `HAL_GPIO_DeInit()`

Les trois fonctions de base

IO operation functions

This section contains the following APIs:

- `HAL_GPIO_ReadPin()`
- `HAL_GPIO_WritePin()`
- `HAL_GPIO_TogglePin()`
- `HAL_GPIO_LockPin()`
- `HAL_GPIO_EXTI_IRQHandler()`

Renvoi SET ou RESET suivant l'état de la pin
met une pin à SET ou RESET
inverse l'état d'une pin

HAL : Couche d'abstraction entre utilisateur et accès aux registres des périphériques

Organisation du code

- dans lib/HAL/
- paire de fichiers .h et .c pour chaque périphérique (ex : stm32f3xx_hal_gpio.{c,h})
- structures de données dans les .h
- fonctions et documentation dans les .c

Prise en main STM32CubeIDE

Utilisation de la bibliothèque HAL (Hardware Abstraction Layer) avec l'interface GPIO



Fonction d'Initialisation d'un port

HAL_GPIO_Init

Function name

```
void HAL_GPIO_Init (GPIO_TypeDef * GPIOx,  
GPIO_InitTypeDef * GPIO_Init)
```

Function description

Initialize the GPIOx peripheral according to the specified parameters in the GPIO_Init.

Parameters

- **GPIOx:** where x can be (A..H) to select the GPIO peripheral for STM32L4 family
- **GPIO_Init:** pointer to a GPIO_InitTypeDef structure that contains the configuration information for the specified GPIO peripheral.

Return values

- **None:**

GPIOx : indiquer le port quand veut initialiser

***GPIO-init** : pointeur contient toutes les informations ce que je veux configurer sur ce point par exemple le numéro de pin

Prise en main STM32CubeIDE

Utilisation de la bibliothèque HAL (Hardware Abstraction Layer) avec l'interface GPIO

Fonction d'écriture dans une broche configurée en sortie

HAL_GPIO_WritePin

Function name **void HAL_GPIO_WritePin (GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin, GPIO_PinState PinState)**

Function description Set or clear the selected data port bit.

Parameters

- **GPIOx:** where x can be (A..H) to select the GPIO peripheral for STM32L4 family
- **GPIO_Pin:** specifies the port bit to be written. This parameter can be one of GPIO_PIN_x where x can be (0..15).

GPIO_Write : pour commander la sortie niveau bas ou haut

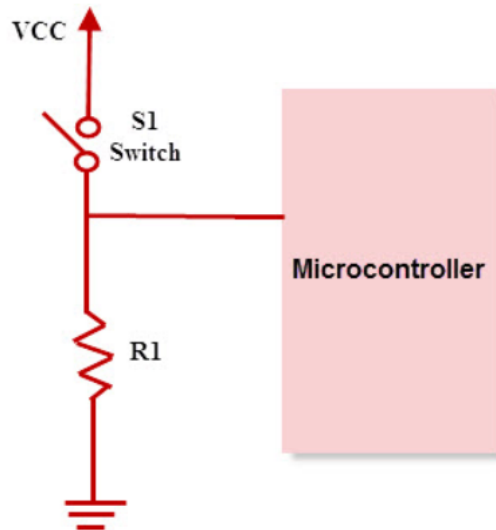
Prise en main STM32CubeIDE



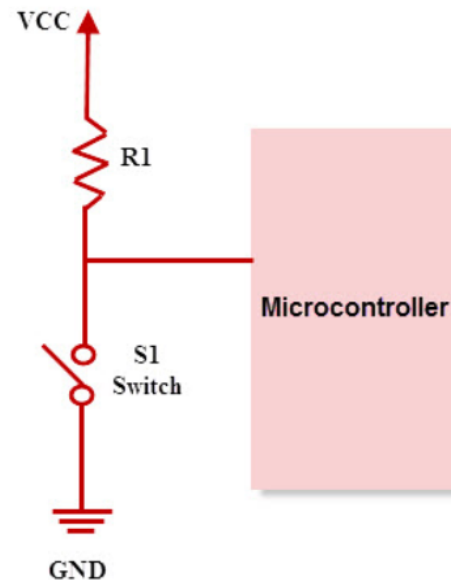
Utilisation de la bibliothèque HAL (Hardware Abstraction Layer) avec l'interface GPIO

Fonction d'écriture dans une broche configurée en sortie

```
// LED ON
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET);
HAL_Delay(100);
// LED OFF
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_RESET);
HAL_Delay(100);
```



Pull-down Resistor



Pull-up Resistor

Prise en main STM32CubeIDE

Utilisation de la bibliothèque HAL (Hardware Abstraction Layer) avec l'interface GPIO

Fonction d'écriture dans une broche configurée en sortie

```
// LED ON
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET);
HAL_Delay(100);
// LED OFF
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_RESET);
HAL_Delay(100);
```

Prise en main STM32CubeIDE

Utilisation de la bibliothèque HAL (Hardware Abstraction Layer) avec l'interface GPIO



Fonction de lecture de l'état d'une broche configurée en entrée

HAL_GPIO_ReadPin

Function name	GPIO_PinState HAL_GPIO_ReadPin (GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin)
Function description	Read the specified input port pin.
Parameters	• GPIOx: where x can be (A..H) to select the GPIO peripheral for STM32L4 family • GPIO_Pin: specifies the port bit to read. This parameter can be GPIO_PIN_x where x can be (0..15).
Return values	• The: input port pin value. Retourne une valeur soit 0 ou 1

Prise en main STM32CubeIDE

Exemple : Allumer la LED quand le bouton poussoir est appuyé



Utilisation de la bibliothèque HAL (Hardware Abstraction Layer) avec l'interface GPIO

Fonction de lecture de l'état d'une broche configurée en entrée

```
// IF Button Is Pressed
if(HAL_GPIO_ReadPin (GPIOA, GPIO_PIN_9))
{
    // Set The LED ON!
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET);
}
else
{
    // Else .. Turn LED OFF!
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_RESET);
}
```

Bouton poussoir brancher sur
pin9

LED brancher sur pin8

Prise en main STM32CubeIDE

Exemple : LED blinking avec REST et SET

```
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1, GPIO_PIN_RESET) ;  
HAL_Delay(500) ;  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1, GPIO_PIN_SET) ;  
HAL_Delay(500) ;
```

Exemple : LED blinking avec 0 et 1 (et Label : LED)

```
HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, 0) ;  
HAL_Delay(500) ;  
HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, 1) ;  
HAL_Delay(500) ;
```

Exemple : LED blinking avec TOGGLE

```
HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_1) ;  
HAL_Delay(500) ;
```

Configuration Entrée/sortie

test.ioc - Pinout & Configuration

Pinout & Configuration | Clock Configuration | Project Manager | Tools

Software Packs | Pinout

GPIO Mode and Configuration

Mode

Configuration

Group By Peripherals

GPIO

Search Signals

Search (Ctrl+F) ☐ Show only Modified Pins

P...	Sign...	GPIO...	GPIO...	GPIO...	Maxi...	User ...	Modif...
PA0...	n/a	Low	Outp...	No p...	Low		☐
PA1	n/a	Low	Outp...	No p...	High	LED[...	☒
PA2	n/a	Low	Outp...	No p...	Low		☐
PA8	n/a	n/a	Input ...	No p...	n/a		☐
PB14	n/a	n/a	Input ...	No p...	n/a		☐
PB15	n/a	n/a	Input ...	No p...	n/a		☐
PC1...	n/a	n/a	Anal...	No p...	n/a		☐

GPIO mode: Output Push Pull

GPIO Pull-up/Pull-down: No pull-up and no pull-down

Maximum output speed: High

User Label: LED[ledgren]

Pinout view | System view

GPIO_Analog

GPIO_Output

LED[ledgren]

GPIO_Output

STM32F401CCUx
UFQFPN48

Unused GPIOs: 29 / 36

```
/* USER CODE END WHILE */

HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin,GPIO_PIN_SET)
HAL_Delay(500)
HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin,GPIO_PIN_RESET)
HAL_Delay(500)

/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

Étapes de configuration des horloges

Pinout & Configuration

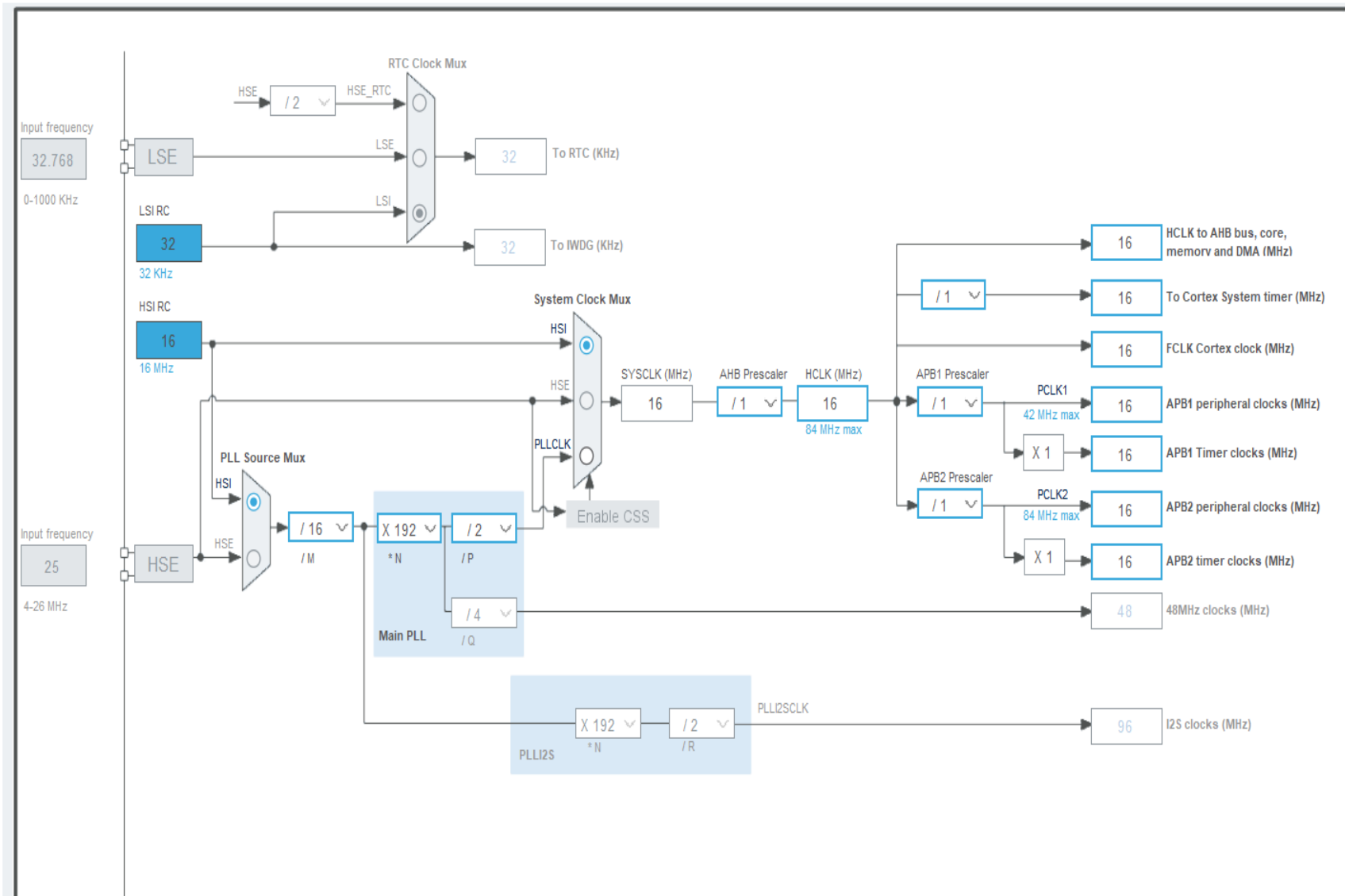
Clock Configuration

Project Manager

Tools



Resolve Clock Issues



Configuration de l'horloge système STM32 dans STM32CubeIDE

Étape 1 : Ouverture du projet

Démarrez STM32CubeIDE et ouvrez votre projet STM32 existant.

Étape 2 : Accès à la configuration des horloges

- Localisez et ouvrez le fichier `.ioc` de votre projet
- Cliquez sur l'onglet "**Clock Configuration**" dans l'interface graphique

Étape 3 : Sélection de la source d'horloge

Déterminez quelle source d'oscillateur utiliser :

- **HSI (High Speed Internal)** : Oscillateur interne, généralement 16 MHz
- **HSE (High Speed External)** : Oscillateur externe (quartz), fréquence variable (typiquement 8 MHz ou 25 MHz)

Étape 4 : Configuration de la PLL (Phase-Locked Loop)

Si vous souhaitez augmenter la fréquence système :

- Activez la PLL
- Configurez les paramètres de multiplication et division
- **Exemple** : HSE de 8 MHz → PLL configurée pour obtenir 168 MHz (selon votre microcontrôleur)

Étape 5 : Configuration des horloges périphériques

Vérifiez et ajustez les fréquences pour chaque bus :

- **AHB** (Advanced High-performance Bus)
- **APB1** et **APB2** (Advanced Peripheral Bus)
- Assurez-vous que les fréquences respectent les limites maximales de chaque périphérique (UART, SPI, I2C, etc.)

Étape 6 : Génération du code

- Cliquez sur l'icône "**Generate Code**" (roue dentée) dans la barre d'outils
- Ou utilisez : **Project** → **Generate Code**

Étape 7 : Vérification du code généré

- Ouvrez le fichier `main.c` dans le dossier `Core/Src`
- Localisez la fonction `SystemClock_Config()`
- Vérifiez que les paramètres correspondent à votre configuration

Introduction au PWM

Qu'est-ce que le PWM ?

La **Modulation de Largeur d'Impulsion (PWM)** est une technique permettant de contrôler la puissance fournie à des composants électroniques en modifiant la durée des impulsions dans un signal périodique.

Principe de fonctionnement

Le PWM génère un signal carré où :

- **Fréquence** : Nombre de cycles par seconde (Hz)
- **Période** : Temps d'un cycle complet ($T = 1/f$)
- **Duty Cycle** : Pourcentage du temps où le signal est à l'état HAUT
- **Amplitude** : Tension maximale du signal (3.3V pour STM32)

Applications du PWM

Applications courantes :

- **Contrôle de moteurs** : Vitesse et direction
- **LED Dimming** : Variation de luminosité
- **Audio** : Synthèse sonore
- **Alimentation** : Régulation de tension

Théorie du PWM - Fondamentaux

Paramètres clés

Formules essentielles :

- Période (T) = $1 / \text{Fréquence (f)}$
- Duty Cycle (%) = $(\text{Temps HAUT} / \text{Période}) \times 100$
- Tension moyenne = $V_{cc} \times \text{Duty Cycle}$

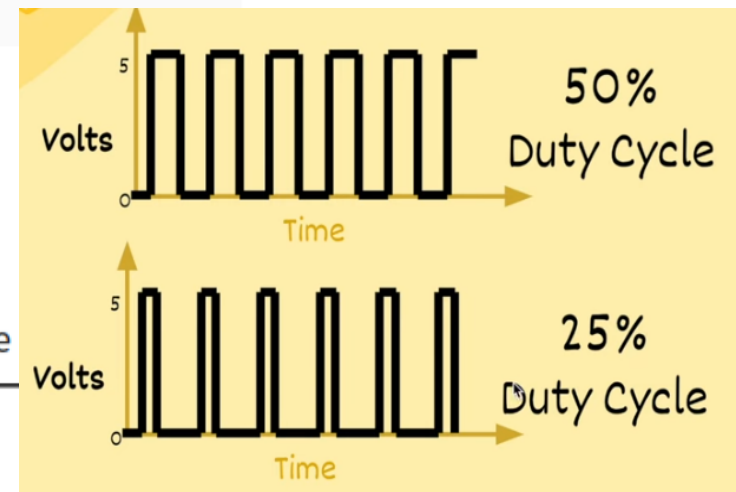
Calculs pratiques

Duty Cycle	100% = V_{cc}	75% = $0.75 \times V_{cc}$	50% = $0.5 \times V_{cc}$	25% = $0.25 \times V_{cc}$	0% = 0V
Tension moyenne (3.3V)	3.3V	2.475V	1.65V	0.825V	0V

 **Astuce importante** : Pour une LED, un duty cycle de 50% ne signifie pas une luminosité à 50%. La perception humaine est logarithmique - vous utiliserez souvent des duty cycles plus faibles pour une luminosité apparemment réduite.

Types de signaux PWM

1. **Fast PWM** : Fréquence élevée, résolution fixe
2. **Phase Correct PWM** : Symétrie autour du point milieu
3. **Frequency and Phase Correct PWM** : Contrôle total fréquence et phase

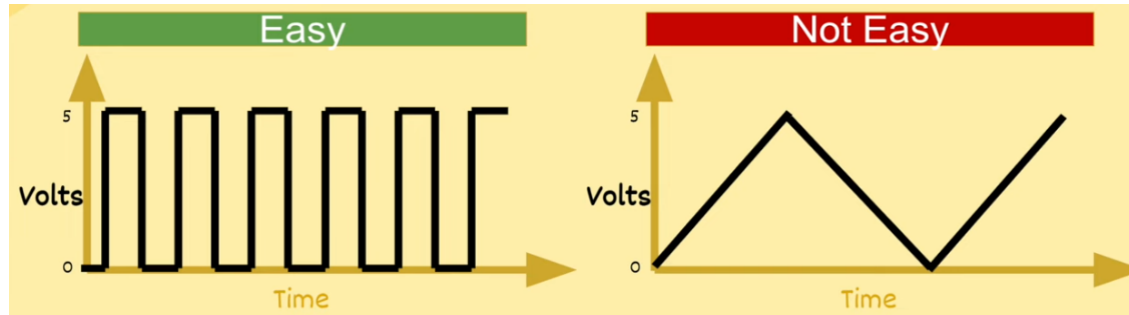


Fake analog

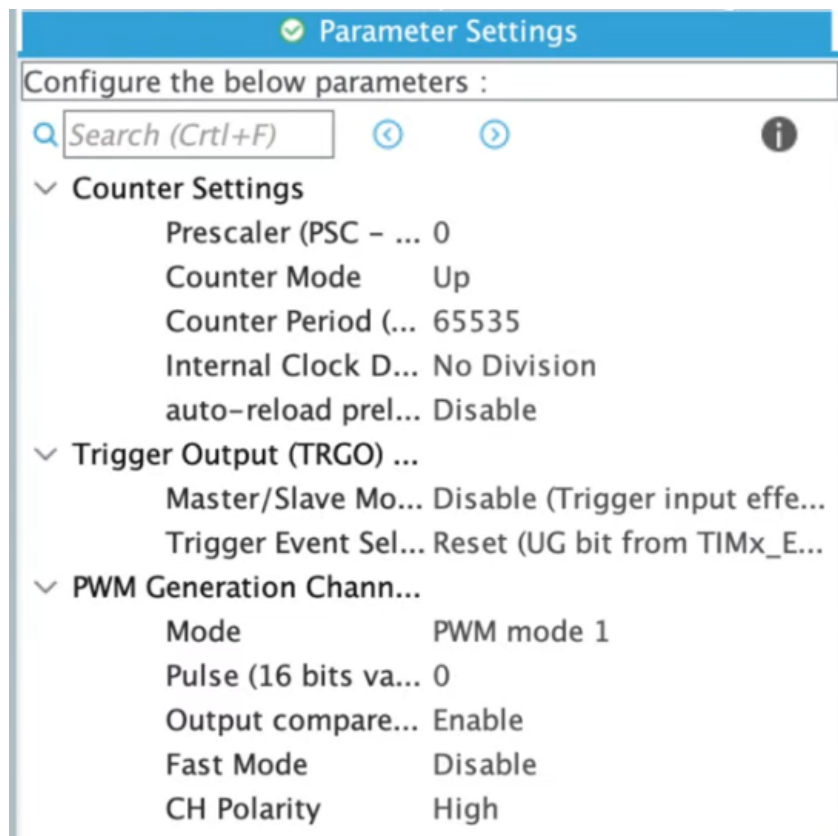
PWM = Pulse Width Modulation

Real analog

DAC = Digital to Analog Converter



STM32



ARDUINO

`analogWrite(pin, dutyCycle)`

↑
0-255

Types de timers disponibles

Type Timer	Résolution	Canaux PWM	Usage recommandé
TIM1/TIM8	16-bit	4	PWM avancé, moteurs
TIM2/TIM3/TIM4/TIM5	16-bit	4	PWM général, LED
TIM6/TIM7	16-bit	Aucun	Base de temps
TIM9/TIM10/TIM11	16-bit	2/1/1	PWM simple

Configuration du prescaler et de l'ARR

Formule de calcul :

- Fréquence PWM = $\text{Frequence_CK} / (\text{Prescaler} + 1) / (\text{ARR} + 1)$
- Résolution = $(\text{ARR} + 1)$ bits



Exemple de calcul :

Données :

- Fréquence PWM souhaitée : 1kHz
- Fréquence horloge : 72MHz (STM32F103)
- Résolution : 8-bit (255 pas)

Calcul :

- $\text{ARR} = 255$ (pour 8-bit)
 - $\text{Prescaler} = (72\text{MHz} / 1\text{kHz} / 256) - 1 = 280$
-

Q 

Categories A->Z

System Core >

Analog >

Timers ▼

RTC

▲ TIM1

▲ TIM2

TIM3

▲ TIM4

▲ TIM5

▲ TIM9

TIM10

TIM11

Connectivity >

Multimedia >

Computing >

Middleware and Software Packs >

TIM5 Mode and Configuration

Mode

Slave Mode Disable ▼

Trigger Source Disable ▼

☐ Internal Clock

Channel1 Disable ▼

Channel2 Disable ▼

Channel3 Disable ▼

Channel4 Output Compare CH4 ▼

Combined Channels Disable ▼

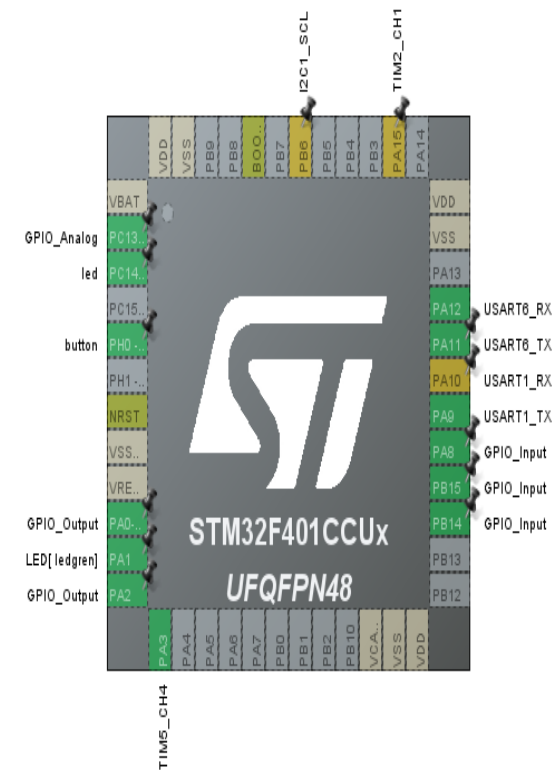
☐ XOR activation☐ One Pulse Mode

Configuration

Reset Configuration

Pinout view

System view



Calculate the PWM frequency

- System Clock is 72MHz: We can “count” 72-million times each second
- 8-bit resolution (0-255): Each cycle is 256 “counts”

$$\frac{72,000,000 \text{ counts}}{1 \text{ second}} \cdot \frac{1 \text{ cycle}}{256 \text{ counts}} = \frac{218250 \text{ cycles}}{1 \text{ second}}$$

Microcontrôleur	Fréquence d'horloge	Architecture
Arduino Uno	16 MHz	AVR 8-bit
STM32F4	168 MHz	ARM Cortex-M4 32-bit

Ratio : STM32F4 = 10.5× plus rapide

Convertisseurs ADC et DAC - STM32F4

1. Vue d'ensemble

Architecture STM32F4

****Microcontrôleur**** : ARM Cortex-M4 168MHz ****Alimentation**** : 1.8V - 3.6V
****Mémoire**** : 1MB Flash + 192KB SRAM

Convertisseurs Intégrés

Convertisseur	Spécifications	Usage
ADC	12-bit SAR, 18 canaux, 36MHz	Lecture capteurs analogiques
DAC	2 canaux 12-bit, sortie buffer	Génération signaux

2. Configuration STM32CubeIDE

Configuration ADC (Pinout & Configuration)

ADC1:

Mode: Independent

Clock Prescaler: PCLK2 div 4

Resolution: 12 bits

Canaux activés:

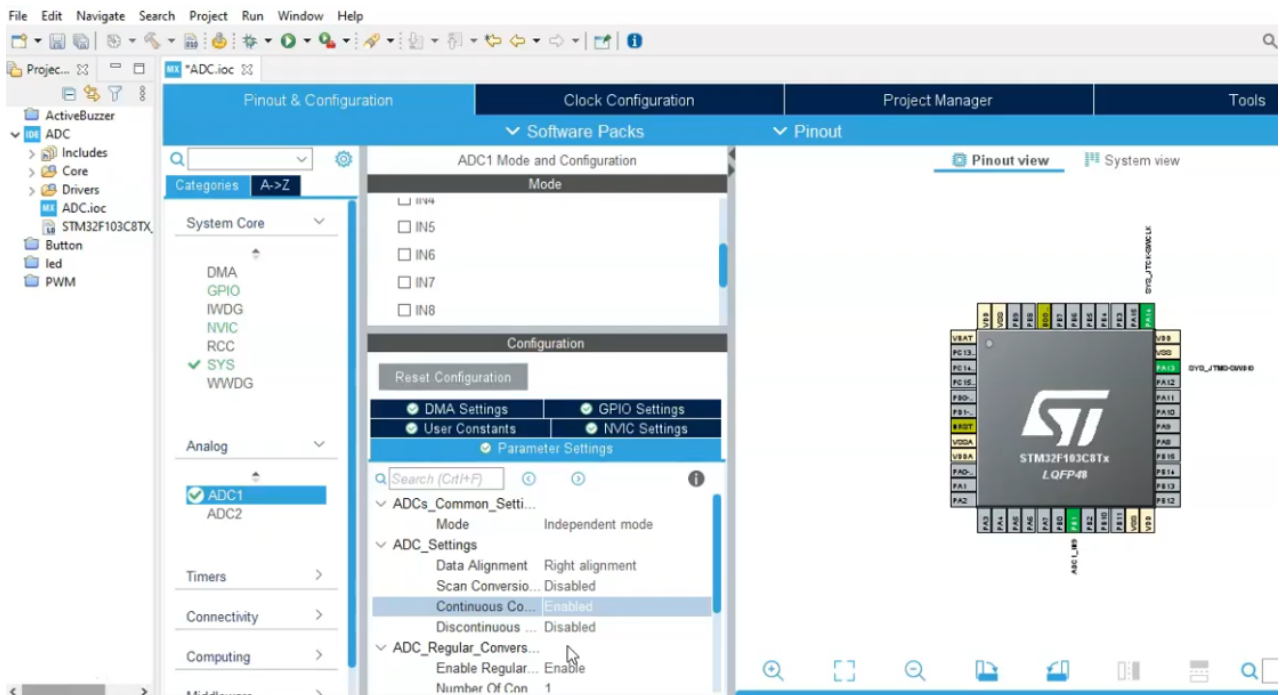
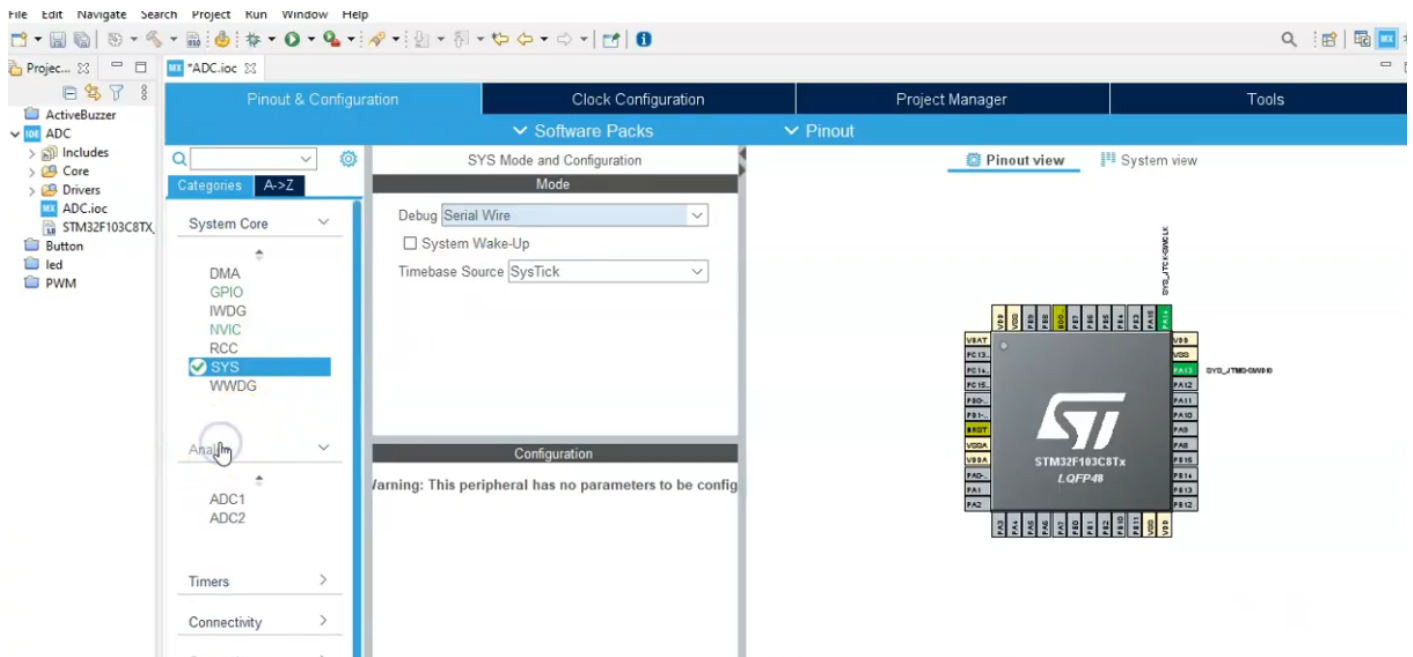
- IN0 → PA0 (Rank 1)
- IN1 → PA1 (Rank 2)
- IN2 → PA2 (Rank 3)
- IN3 → PA3 (Rank 4)

DMA:

Mode: Normal

Direction: Peripheral to Memory

Priority: High





Configuration DAC

DAC1:

Channel 1 (PA4):

Trigger: None

Output Buffer: Enable

Channel 2 (PA5):

Trigger: Timer 6 TRGO

Output Buffer: Enable

Timer 6:

Prescaler: 83 (1MHz)

Period: 999 (1kHz)



Code généré automatiquement

```
// Variables globales
ADC_HandleTypeDef hadc1;
DAC_HandleTypeDef hdac;
DMA_HandleTypeDef hdma_adc1, hdma_dac1;
TIM_HandleTypeDef htim6;

#define ADC_CHANNELS 4
uint32_t adc_readings[ADC_CHANNELS];
uint16_t dac_output = 2048; // Valeur milieu de gamme
```

3. ADC - Convertisseur Analogique-Numérique



Principe de fonctionnement

****Conversion**** : Tension → Valeur numérique ****Formule**** : `Valeur = (Vin / Vref) × 4095` ****Résolution**** : 12 bits = 4096 valeurs (0-4095)

Initialisation ADC

```
void ADC_Init(void)
{
    ADC_ChannelConfTypeDef sConfig = {0};

    hadc1.Instance = ADC1;
    hadc1.Init.ClockPrescaler = ADC_CLOCK_SYNC_PCLK_DIV4;
    hadc1.Init.Resolution = ADC_RESOLUTION_12B;
    hadc1.Init.ScanConvMode = ENABLE; // Multi-canaux
    hadc1.Init.ContinuousConvMode = DISABLE;
    hadc1.Init.NbrOfConversion = ADC_CHANNELS;
    hadc1.Init.DMAContinuousRequests = ENABLE;

    HAL_ADC_Init(&hadc1);

    // Configuration des 4 canaux
    for(int i = 0; i < ADC_CHANNELS; i++)
    {
        sConfig.Channel = i; // Channel 0,1,2,3
        sConfig.Rank = i + 1;
        sConfig.SamplingTime = ADC_SAMPLETIME_3CYCLES;
        HAL_ADC_ConfigChannel(&hadc1, &sConfig);
    }
}
```



Lecture ADC avec callback

```
// Callback appelé à la fin de chaque conversion
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
{
    if(hadc->Instance == ADC1)
    {
        // adc_readings[0] = Tension sur PA0
        // adc_readings[1] = Tension sur PA1
        // adc_readings[2] = Tension sur PA2
        // adc_readings[3] = Tension sur PA3

        // Conversion en volts
        for(int i = 0; i < ADC_CHANNELS; i++)
        {
            float voltage = (float)adc_readings[i] * 3.3f / 4095.0f;
            printf("Canal %d: %.3fV (%lu)\n", i, voltage,
adc_readings[i]);
        }
    }
}

// Démarrer acquisition
void ADC_Start_Reading(void)
{
    HAL_ADC_Start_DMA(&hadc1, adc_readings, ADC_CHANNELS);
}
```

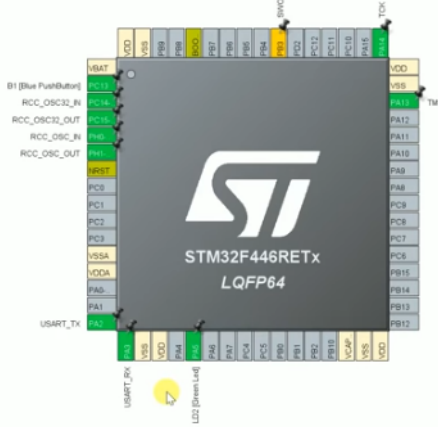
4. DAC - Convertisseur Numérique-Analogique



Principe de fonctionnement

****Conversion**** : Valeur numérique → Tension ****Formule**** : $V_{out} = (DAC_Value / 4095) \times V_{ref}$ ****Sorties**** : PA4 (Canal 1), PA5 (Canal 2)

}



Génération de signaux

```
// Génération onde sinus
void DAC_Generate_Sine(void)
{
    static uint16_t sine_table[256];

    // Table sinus pré-calculée
    for(int i = 0; i < 256; i++)
    {
        float angle = 2.0f * M_PI * i / 256.0f;
        sine_table[i] = (uint16_t)((sin(angle) + 1.0f) * 2047.5f);
    }

    // Lecture table sinus
    static uint8_t index = 0;
    HAL_DAC_SetValue(&hdac, DAC_CHANNEL_1, DAC_ALIGN_12B_R,
sine_table[index]);
    index = (index + 1) % 256;
}

// Valeur directe (contrôle manuel)
void DAC_Set_Voltage(uint16_t value)
{
    // Limiter entre 0 et 4095
    if(value > 4095) value = 4095;

    HAL_DAC_SetValue(&hdac, DAC_CHANNEL_1, DAC_ALIGN_12B_R, value);
}
```

5. Application Pratique

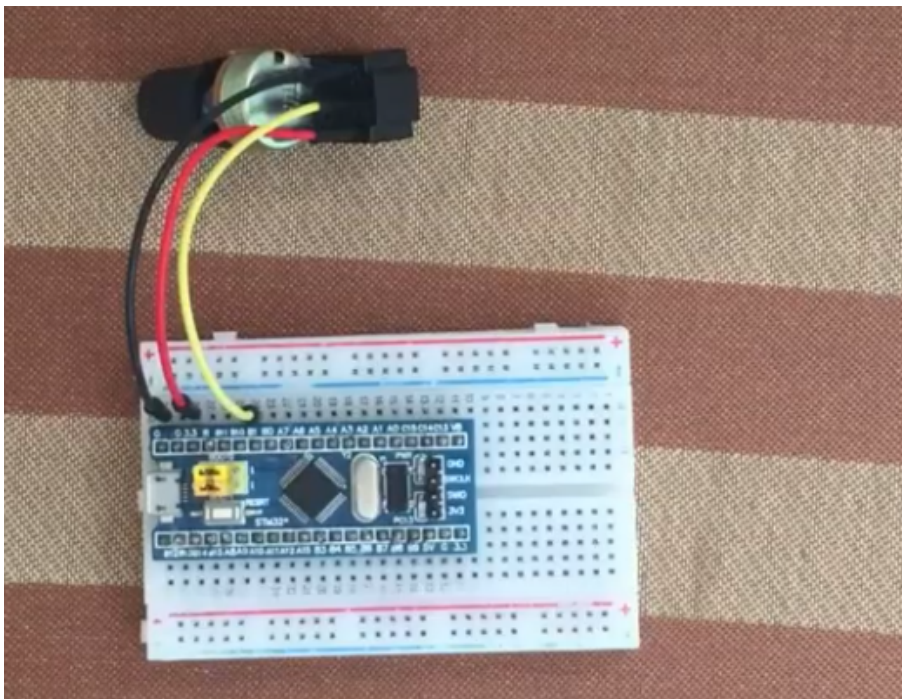
Contrôle Moteur avec Potentiomètre

Objectif

Contrôler la vitesse d'un moteur DC avec un potentiomètre via ADC, afficher la tension de sortie via DAC

Connexions

Composant	Pin STM32F4	Fonction
Potentiomètre 10k Ω	PA0 (ADC1_IN0)	Commande vitesse
Moteur DC	PB6 (TIM4_CH1)	Sortie PWM
LED	PA4 (DAC_OUT1)	Visualisation tension



Code complet

```
#include "stm32f4xx_hal.h"

// Variables globales
ADC_HandleTypeDef hadc1;
DAC_HandleTypeDef hdac;
TIM_HandleTypeDef htim4; // Pour PWM moteur
TIM_HandleTypeDef htim6; // Pour DAC

uint32_t adc_value = 0;
uint16_t motor_speed = 0;

// Callbacks
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
{
    if(hadc->Instance == ADC1)
    {
        // Lire valeur potentiomètre (0-4095)
        adc_value = adc_readings[0];

        // Convertir en vitesse moteur (0-100%)
        motor_speed = (uint16_t)((float)adc_value * 100.0f / 4095.0f);

        // Appliquer vitesse moteur (PWM)
        __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_1, motor_speed * 10);

        // Afficher tension équivalente sur DAC
        HAL_DAC_SetValue(&hdac, DAC_CHANNEL_1, DAC_ALIGN_12B_R,
adc_value);

        printf("Potentiomètre: %lu (%.2f%%) | DAC: %lu (%.2fV)\n",
            adc_value,
            (float)adc_value * 100.0f / 4095.0f,
            adc_value,
            (float)adc_value * 3.3f / 4095.0f);
    }
}

int main(void)
{
    HAL_Init();
    SystemClock_Config();
```



```

// Initialisations
MX_GPIO_Init();
MX_ADC1_Init();
MX_DAC_Init();
MX_TIM4_Init(); // PWM moteur
MX_TIM6_Init(); // Timer DAC
MX_DMA_Init();

// Démarrer périphériques
HAL_ADC_Start_DMA(&hadc1, adc_readings, 1);
HAL_DAC_Start(&hdac, DAC_CHANNEL_1);
HAL_TIM_PWM_Start(&htim4, TIM_CHANNEL_1);
HAL_TIM_Base_Start(&htim6);

while(1)
{
    HAL_Delay(500); // Actualisation 2Hz

    // Lecture directe potentiomètre (alternative)
    // HAL_ADC_Start(&hadc1);
    // HAL_ADC_PollForConversion(&hadc1, 100);
    // uint16_t value = HAL_ADC_GetValue(&hadc1);
}
}

```

Configuration Timer PWM

```

void MX_TIM4_Init(void)
{
    TIM_OC_InitTypeDef sConfigOC = {0};

    htim4.Instance = TIM4;
    htim4.Init.Prescaler = 83;           // 1MHz
    htim4.Init.CounterMode = TIM_COUNTERMODE_UP;
    htim4.Init.Period = 999;             // 1kHz PWM
    htim4.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;

    HAL_TIM_PWM_Init(&htim4);

    sConfigOC.OCMode = TIM_OCMODE_PWM1;
    sConfigOC.Pulse = 0;                 // Vitesse initiale
    sConfigOC.OCpolarity = TIM_OCPOLARITY_HIGH;
    HAL_TIM_PWM_ConfigChannel(&htim4, &sConfigOC, TIM_CHANNEL_1);
}

```

Exercices Pratiques STM32

1

Clignotement LED avec Variations

Facile

Description : Créez un programme qui fait clignoter une LED avec trois vitesses différentes : lente (2 secondes), moyenne (500ms), et rapide (100ms).

✓ Objectifs d'Apprentissage

- Maîtriser HAL_GPIO_TogglePin()
- Utiliser HAL_Delay() avec différentes valeurs
- Comprendre la structure de la boucle while(1)
- Configurer correctement les GPIO en mode sortie

📅 Matériel Requis

- 1× LED (ou LED intégrée sur PA5)
- 1× Résistance 220Ω (si LED externe)
- Carte STM32

📋 Spécifications Techniques

- Pin LED : GPIOA, Pin 5
- Mode : GPIO_MODE_OUTPUT_PP
- Séquence : 5 clignotements lents → 10 moyens → 20 rapides → recommencer

2

Feu de Circulation (Traffic Light)

Facile

Description : Simulez un feu de circulation avec 3 LEDs (Rouge, Jaune, Verte) qui s'allument selon un cycle réaliste.

✓ Objectifs d'Apprentissage

- Contrôler plusieurs GPIO simultanément
- Utiliser `HAL_GPIO_WritePin()` pour états multiples
- Créer une séquence temporelle logique
- Gérer l'état de plusieurs sorties

📅 Matériel Requis

- 3× LEDs (Rouge, Jaune, Verte)
- 3× Résistances 220Ω
- Breadboard et câbles de connexion

📋 Spécifications Techniques

- LED Rouge : GPIOA, Pin 0
- LED Jaune : GPIOA, Pin 1
- LED Verte : GPIOA, Pin 2
- **Séquence :** Rouge (5s) → Rouge+Jaune (2s) → Vert (5s) → Jaune (2s) → recommencer

💡 Indice

Créez une fonction pour éteindre toutes les LEDs avant chaque changement d'état. Utilisez l'opérateur OR (`|`) pour éteindre plusieurs pins simultanément.

3

Compteur de Pressions de Bouton

Moyen

Description : Créez un compteur qui s'incrémente à chaque pression de bouton et affiche le compte via des LEDs en binaire (4 bits).

☑ Objectifs d'Apprentissage

- Lire l'état d'un bouton avec `HAL_GPIO_ReadPin()`
- Implémenter un anti-rebond logiciel
- Convertir un nombre décimal en binaire
- Contrôler 4 LEDs pour affichage binaire
- Détecter les fronts descendants (pression)

📦 Matériel Requis

- 1× Bouton poussoir
- 4× LEDs
- 4× Résistances 220Ω
- 1× Résistance $10k\Omega$ (pull-up pour bouton)

📋 Spécifications Techniques

- Bouton : GPIOC, Pin 13 (pull-up interne)
- LED Bit 0 (LSB) : GPIOA, Pin 0
- LED Bit 1 : GPIOA, Pin 1
- LED Bit 2 : GPIOA, Pin 2
- LED Bit 3 (MSB) : GPIOA, Pin 3
- Compteur : 0-15 (4 bits), retour à 0 après 15

4

Chenillard (Knight Rider Effect)

Moyen

Description : Créez un effet de chenillard avec 8 LEDs : une LED se déplace de gauche à droite puis de droite à gauche en boucle.

✓ Objectifs d'Apprentissage

- Utiliser des tableaux pour gérer plusieurs GPIO
- Implémenter des boucles bidirectionnelles
- Créer des animations LED
- Optimiser le code avec des fonctions

📅 Matériel Requis

- 8× LEDs
- 8× Résistances 220Ω
- Breadboard et câbles

📋 Spécifications Techniques

- 8 LEDs connectées sur GPIOB Pins 0-7
- Délai entre chaque mouvement : 100ms
- Séquence : 0→1→2→3→4→5→6→7→6→5→4→3→2→1→0 (boucle)

💡 Indice

Créez un tableau constant avec les pins GPIO. Utilisez deux boucles for : une qui compte de 0 à 7, et une autre qui décrémente de 6 à 1.